# Team Note of Planche Team

MinhhHoangPersia - Pain - Lingg

Compiled on October 29, 2026

## Contents


**1 Template** 1
- 1.1 Random . . . 1
- 1.2 Fast IO . . . 1
- 1.3 com.sh . . . 1
- 1.4 check.cpp (Linux) . . . 1
- 1.5 com.bat . . . 2
- 1.6 check.cpp (Window) . . . 2

**2 Data Structure** 2
- 2.1 Segment Tree Walk . . . 2
- 2.2 Persistent Segment Tree . . . 2
- 2.3 Trie . . . 3
- 2.4 Persistent Trie . . . 3
- 2.5 Fenwick Tree . . . 4
- 2.6 Fenwick Tree 2D . . . 4
- 2.7 XOR Basis . . . 5
- 2.8 Bipartite DSU Rollback . . . 5
- 2.9 Ordered Set . . . 6
- 2.10 Line Container . . . 7
- 2.11 Deque CHT . . . 7

**3 Graphs** 8
- 3.1 2 SAT . . . 8
- 3.2 LCA O(1) . . . 8
- 3.3 Centroid Decomposition . . . 9
- 3.4 Heavy Light Decomposition . . . 9
- 3.5 Bellman Ford . . . 9
- 3.6 Dinic . . . 10
- 3.7 Flow with lower bound . . . 11
- 3.8 Min Cost Max Flow . . . 12
- 3.9 Dsu On Tree(Sack) . . . 13
- 3.10 MO On Tree . . . 13
- 3.11 Dynamic Connectivity . . . 14

**4 DP** 15
- 4.1 DP Digit Optimization 1 . . . 15
- 4.2 DP Digit Optimization 2 . . . 15
- 4.3 DP SoS . . . 16
- 4.4 DP Divide And Conquer . . . 16

**5 Strings** 16
- 5.1 KMP . . . 16
- 5.2 Z Function . . . 17
- 5.3 Suffix Array . . . 17
- 5.4 Aho Corasick . . . 18

**6 Math** 19
- 6.1 Extended Euclid . . . 19
- 6.2 CRT . . . 19
- 6.3 Lucas Theorem . . . 20
- 6.4 Euler Totient . . . 20
- 6.5 Mobius Function . . . 21
- 6.6 Miller Rabin . . . 21

**7 Geometry** 22
- 7.1 Geomtry Point . . . 22
- 7.2 Convex Hull . . . 23
- 7.3 Manhattan MST . . . 23
- 7.4 Common Geometry Operations . . . 23

**8 Miscellaneous** 24
- 8.1 Alien Trick . . . 24
- 8.2 Interval Container . . . 24
- 8.3 Ternary Search . . . 25


# 1 Template

## 1.1 Random

```
mt19937
rng(chrono::steady_clock::now().time_since_epoch().count());
ll rnd(ll l, ll r) {
  return uniform_int_distribution<ll>(l, r)(rng);
}
```

## 1.2 Fast IO

```
ios_base::sync_with_stdio(false);
cin.tie(nullptr);
```

## 1.3 com.sh

```
#!/bin/bash
set -e

name="${1%.cpp}"

g++ -std=c++17 -O2 -pipe "$name.cpp" -o "$name"

# Cap quyen: chmod +x com.sh
# Xem diff: diff -u file1 file2
# Do time chay: time ./main < TestInput
# Gioi han time chay: timeout 2s ./main < TestInput
```

## 1.4 check.cpp (Linux)

```
#include <bits/stdc++.h>
using namespace std;
#define bit(i, x) (x >> i & 1)
#define ll long long
const int N = 2e5 + 5;
const int mod = 998244353;

int findFirstDifferentLine(string file1, string file2) {
  ifstream a(file1), b(file2);
  string x, y;
  int line = 1;

  while(true) {
    bool ok1 = (bool)getline(a, x);
    bool ok2 = (bool)getline(b, y);

    if(!ok1 && !ok2) return -1;
    if(ok1 != ok2 || x != y) return line;
    line++;
  }
}

signed main(int argc, char* argv[]) {
  ios_base::sync_with_stdio(0);
  cin.tie(0); cout.tie(0);

  string sol = "main";
  string brute = "brute";
  string gen = "gen";

  for(string name : {sol, brute, gen}) {
    if(system(("./com.sh " + name).c_str()) != 0) {
      cout << "Compilation failed: " << name << '\n';
      return 1;
    }
  }

  int test = 20;
```

```
  for(int i = 1; i <= test; i++) {
    if(system(("./" + gen + " > TestInput").c_str()) != 0 ||
       system(("./" + sol + " < TestInput >
       MainSolution_Output").c_str()) != 0 ||
       system(("./" + brute + " < TestInput >
       BruteSolution_Output").c_str()) != 0) {
      cout << "Runtime error at test " << i << '\n';
      return 1;
    }

    int val = system("diff -q MainSolution_Output
    BruteSolution_Output > /dev/null");

    cout << "Test " << i << ": ";
    if(val == 0) {
      cout << "true";
    }
    else {
      cout << "FALSE\n";
      cout << "WRONG ANSWER, FAILED AT TEST CASE " << i <<
      '\n';
      int line = findFirstDifferentLine("MainSolution_Output",
      "BruteSolution_Output");
      cout << "First different at line " << line << '\n';
      system("diff -u MainSolution_Output
      BruteSolution_Output");
      return 0;
    }
    cout << '\n';
  }

  cout << "Passed! All test cases are correct!\n";

  return 0 ^ 0;
}
```

### 1.5 com.bat

```
@echo off
g++ -std=c++17 -O2 -Wl,--stack,536870912 "%~1.cpp" -o
"%~1.exe"
```

### 1.6 check.cpp (Window)

```
#include <bits/stdc++.h>
using namespace std;
#define bit(i, x) (x >> i & 1)
#define ll long long
const int N = 2e5 + 5;
const int mod = 998244353;

int findFirstDifferentLine(string file1, string file2) {
  ifstream a(file1, ios::binary), b(file2, ios::binary);
  int line = 1;

  while(true) {
    int x = a.get(), y = b.get();
    if(x != y) return line;
    if(x == EOF) return -1;
    if(x == '\n') line++;
  }
}

signed main(int argc, char* argv[]) {
  ios_base::sync_with_stdio(0);
  cin.tie(0); cout.tie(0);

  string sol = "main";
  string brute = "brute";
  string gen = "gen";

  for(string name : {sol, brute, gen}) {
    if(system(("call com.bat " + name).c_str()) != 0) {
      cout << "Compilation failed: " << name << '\n';
      return 1;
    }
  }

  int test = 20;
  if(argc > 1) test = atoi(argv[1]);

  for(int i = 1; i <= test; i++) {
    if(system((gen + ".exe > TestInput").c_str()) != 0 ||
       system((sol + ".exe < TestInput >
       MainSolution_Output").c_str()) != 0 ||
       system((brute + ".exe < TestInput >
       BruteSolution_Output").c_str()) != 0) {
      cout << "Runtime error at test " << i << '\n';
      return 1;
    }

    int val = system("fc /b MainSolution_Output
    BruteSolution_Output > nul");

    cout << "Test " << i << ": ";
    if(val == 0) {
      cout << "true";
    }
    else {
      if(val != 1) {
        cout << "File comparison failed\n";
        return 1;
      }
      cout << "FALSE\n";
      cout << "WRONG ANSWER, FAILED AT TEST CASE " << i <<
      '\n';
      int line = findFirstDifferentLine("MainSolution_Output",
      "BruteSolution_Output");
      cout << "First different at line " << line << '\n';
      system("fc MainSolution_Output BruteSolution_Output");
      return 1;
    }
    cout << '\n';
  }

  cout << "Passed! All test cases are correct!\n";
  return 0;
}
/*
com check
check
check 1000
*/
```

## 2 Data Structure

### 2.1 Segment Tree Walk

```
/* Find first element in range [u, v] have value >= val */
int findPos(int id, int l, int r, int u, int v, ll val) {
  if(l > v || r < u) return n + 1;
  if(st[id] < val) return n + 1;
  if(l == r) return l;
  push(id);
  int mid = (l + r) / 2;
  int res = findPos(id * 2, l, mid, u, v, val);
  if(res == n + 1) {
    res = findPos(id * 2 + 1, mid + 1, r, u, v, val);
  }
  return res;
}
```

### 2.2 Persistent Segment Tree

```
const int N = 3e5 + 5;
const int LOG = 20;
const int MAX_NODES = N * 40;

struct Node {
  int left, right, v;
  Node(int _left = 0, int _right = 0, int _v = 0) {
    left = _left;
    right = _right;
    v = _v;
  }
} st[MAX_NODES];

int cnt = 0;
int ver[N], nver = 0;
```

```
void refine(int id) {
  st[id].v = st[st[id].left].v + st[st[id].right].v;
}

int build(int l, int r) {
  if (l == r) {
    cnt++;
    st[cnt] = Node(0, 0, 0);
    return cnt;
  }
  int mid = (l + r) >> 1;
  int cur = ++cnt;
  st[cur].left = build(l, mid);
  st[cur].right = build(mid + 1, r);
  refine(cur);
  return cur;
}

int update(int id, int l, int r, int u, int x) {
  if (l == r) {
    cnt++;
    st[cnt] = Node(0, 0, st[id].v + x);
    return cnt;
  }
  int mid = (l + r) >> 1;
  int cur = ++cnt;
  if (u <= mid) {
    st[cur].left = update(st[id].left, l, mid, u, x);
    st[cur].right = st[id].right;
  } else {
    st[cur].left = st[id].left;
    st[cur].right = update(st[id].right, mid + 1, r, u, x);
  }
  refine(cur);
  return cur;
}

int get(int id, int l, int r, int u, int v) {
  if (l > v || r < u) return 0;
  if (l >= u && r <= v) return st[id].v;
  int mid = (l + r) >> 1;
  return get(st[id].left, l, mid, u, v) + get(st[id].right,
  mid + 1, r, u, v);
}

void add_version(int l, int r, int u, int x) {
  nver++;
  ver[nver] = update(ver[nver - 1], l, r, u, x);
}
```

## 2.3 Trie

```
const int LOG_MAX = 29;
const int MAX_NODES = 3e5 * 32;

struct Node {
  int c[2];
  int cnt;

  Node() {
    c[0] = c[1] = 0;
    cnt = 0;
  }
};

Node trie[MAX_NODES];
int node_cnt = 0;

void init_trie() {
  for (int i = 0; i <= node_cnt; i++) {
    trie[i] = Node();
  }
  node_cnt = 0;
}

void insert(int val) {
  int cur = 0;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    if (trie[cur].c[x] == 0) {
      trie[cur].c[x] = ++node_cnt;
    }
    cur = trie[cur].c[x];
    trie[cur].cnt++;
  }
}

bool search(int val) {
  int cur = 0;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    cur = trie[cur].c[x];
    if (cur == 0 || trie[cur].cnt == 0) {
      return false;
    }
  }
  return trie[cur].cnt > 0;
}

int count_occurences(int val) {
  int cur = 0;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    cur = trie[cur].c[x];
    if (cur == 0 || trie[cur].cnt == 0) {
      return 0;
    }
  }
  return trie[cur].cnt;
}

void erase(int val) {
  if (!search(val)) return;
  int cur = 0;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    cur = trie[cur].c[x];
    trie[cur].cnt--;
  }
}

void update(int old_val, int new_val) {
  if (!search(old_val)) return;
  erase(old_val);
  insert(new_val);
}
```

## 2.4 Persistent Trie

```
const int LOG_MAX = 29;
const int N = 1e5 + 5;
const int MAX_NODES = N * 32;

int t[MAX_NODES][2];
int num[MAX_NODES];
int root[N];
int cnt = 0;

void init_trie() {
  for (int i = 0; i <= cnt; i++) {
    t[i][0] = t[i][1] = num[i] = 0;
  }
  cnt = 0;
  root[0] = 0;
}

int newnode() {
  cnt++;
  t[cnt][0] = t[cnt][1] = num[cnt] = 0;
  return cnt;
}

int add_version(int prev_root, int val) {
  int cur_root = newnode();
  int cur = cur_root;
  int old = prev_root;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    t[cur][0] = t[old][0];
    t[cur][1] = t[old][1];
    t[cur][x] = newnode();
    cur = t[cur][x];
```

```
    old = t[old][x];
    t[cur][0] = t[old][0];
    t[cur][1] = t[old][1];
    num[cur] = num[old] + 1;
  }
  return cur_root;
}

// 4. Query Maximum XOR with 'val' using combinations of
versions.
// Example: Query on a tree path from u to v with LCA p,
parent of p is pp.
// The effective count of a node is: num[x] + num[y] - num[p]
- num[pp].
int query_max_xor(int rootX, int rootY, int rootP, int rootPP,
int val) {
  int res = 0;
  int x = rootX, y = rootY, p = rootP, pp = rootPP;
  for (int i = LOG_MAX; i >= 0; i--) {
    int v = (val >> i) & 1;
    int count_opposite = num[t[x][v ^ 1]] + num[t[y][v ^ 1]]
                       - num[t[p][v ^ 1]] - num[t[pp][v ^ 1]];
    if (count_opposite > 0) {
      res |= (1 << i);
      x = t[x][v ^ 1];
      y = t[y][v ^ 1];
      p = t[p][v ^ 1];
      pp = t[pp][v ^ 1];
    } else {
      x = t[x][v];
      y = t[y][v];
      p = t[p][v];
      pp = t[pp][v];
    }
  }
  return res;
}

/*
   USAGE EXAMPLE (Max XOR in an array from L to R):

   // 1. Build prefix versions
   init_trie();
   for (int i = 1; i <= n; i++) {
     root[i] = add_version(root[i - 1], a[i]);
   }

   // 2. Query Max XOR with 'val' in subarray [L, R]
   // For arrays, the effective tree is root[R] - root[L-1].
   // You can use a modified query function that only takes 2
   roots:
   // count_opposite = num[t[rootR][v ^ 1]] -
   num[t[rootL_minus_1][v ^ 1]];
*/
```

## 2.5 Fenwick Tree

```
struct Fenwick {
  int n;
  vector<long long> bit;

  Fenwick(int _n) {
    n = _n;
    bit.assign(n + 1, 0);
  }

  void update(int idx, long long delta) {
    for (; idx <= n; idx += idx & -idx) {
      bit[idx] += delta;
    }
  }

  long long query(int idx) {
    long long sum = 0;
    for (; idx > 0; idx -= idx & -idx) {
      sum += bit[idx];
    }
    return sum;
  }

  long long query(int l, int r) {
    if (l > r) return 0;
    return query(r) - query(l - 1);
  }

  // O(log N) Bit Jumping: Finds the FIRST index where prefix
  sum >= val.
  // Note: This only works if all elements in the BIT are
  non-negative!
  int lower_bound(long long val) {
    if (val <= 0) return 1;
    int pos = 0;
    long long sum = 0;
    for (int i = __lg(n); i >= 0; i--) {
      int next_pos = pos + (1 << i);
      if (next_pos <= n && sum + bit[next_pos] < val) {
        sum += bit[next_pos];
        pos = next_pos;
      }
    }
    // pos is the largest index such that prefix sum < val
    // So pos + 1 is the first index such that prefix sum >=
    val
    return pos + 1;
  }
};
```

## 2.6 Fenwick Tree 2D

```
struct Fenwick2D {
  int n, m;
  vector<vector<long long>> bit1, bit2, bit3, bit4;

  Fenwick2D(int _n, int _m) {
    n = _n;
    m = _m;
    bit1.assign(n + 2, vector<long long>(m + 2, 0));
    bit2.assign(n + 2, vector<long long>(m + 2, 0));
    bit3.assign(n + 2, vector<long long>(m + 2, 0));
    bit4.assign(n + 2, vector<long long>(m + 2, 0));
  }

  void add(int x, int y, long long val) {
    for (int i = x; i <= n; i += i & -i) {
      for (int j = y; j <= m; j += j & -j) {
        bit1[i][j] += val;
        bit2[i][j] += val * x;
        bit3[i][j] += val * y;
        bit4[i][j] += val * x * y;
      }
    }
  }

  void update(int x1, int y1, int x2, int y2, long long val) {
    add(x1, y1, val);
    add(x2 + 1, y1, -val);
    add(x1, y2 + 1, -val);
    add(x2 + 1, y2 + 1, val);
  }

  // Internal Prefix Query: gets the sum of submatrix from (1,
  1) to (x, y)
  long long query_prefix(int x, int y) {
    long long res = 0;
    for (int i = x; i > 0; i -= i & -i) {
      for (int j = y; j > 0; j -= j & -j) {
        res += bit1[i][j] * (x + 1) * (y + 1)
             - bit2[i][j] * (y + 1)
             - bit3[i][j] * (x + 1)
             + bit4[i][j];
      }
    }
    return res;
  }

  // Range Query: gets the sum of submatrix from (x1, y1) to
  (x2, y2)
  long long query(int x1, int y1, int x2, int y2) {
    return query_prefix(x2, y2)
         - query_prefix(x1 - 1, y2)
         - query_prefix(x2, y1 - 1)
         + query_prefix(x1 - 1, y1 - 1);
```

```
  }
};
```

## 2.7 XOR Basis

```
/*
   - Represents a set of numbers using a minimal set of
   linearly independent
     numbers (basis).
   - Any XOR sum of a subset in the original set can be formed
   by a XOR sum
     of a subset of the basis.
   - The size of the basis never exceeds the number of bits
   (LOG_D).
*/
const int LOG_D = 60; // 60 for values up to 10^18, 30 for
values up to 10^9

struct XorBasis {
  long long basis[LOG_D];
  int sz; // Number of elements currently in the basis
  vector<long long> reduced_basis; // Stores the canonical
  form for K-th queries

  XorBasis() {
    memset(basis, 0, sizeof(basis));
    sz = 0;
  }

  // 1. Insert a value. Returns true if successfully inserted
  bool insert(long long x) {
    for (int i = LOG_D - 1; i >= 0; i--) {
      if ((x >> i) & 1) {
        if (!basis[i]) {
          basis[i] = x;
          sz++;
          return true;
        }
        x ^= basis[i];
      }
    }
    return false; // x became 0, meaning it was already
    spanned by the basis
  }

  // 2. Check if a value can be formed by XORing a subset of
  the basis
  bool check(long long x) {
    for (int i = LOG_D - 1; i >= 0; i--) {
      if ((x >> i) & 1) {
        if (!basis[i]) return false;
        x ^= basis[i];
      }
    }
    return true;
  }

  // 3. Get the maximum possible XOR sum using the basis.
  long long get_max(long long start_val = 0) {
    long long res = start_val;
    for (int i = LOG_D - 1; i >= 0; i--) {
      if ((res ^ basis[i]) > res) {
        res ^= basis[i];
      }
    }
    return res;
  }

  // 4. Merge this basis with another basis
  void merge(const XorBasis& other) {
    for (int i = 0; i < LOG_D; i++) {
      if (other.basis[i]) {
        insert(other.basis[i]);
      }
    }
  }

  // K-TH LARGEST / SMALLEST QUERIES
  // Note: These functions only consider DISTINCT values
  generated by the basis.
  // There are exactly 2^sz distinct values (including 0 from
  the empty subset).

  // 5. Build Reduced Row Echelon Form (Canonical Form)
  // MUST be called after all inserts and BEFORE querying k-th
  values!
  void build() {
    reduced_basis.clear();
    for (int i = LOG_D - 1; i >= 0; i--) {
      for (int j = i - 1; j >= 0; j--) {
        // If the j-th bit of basis[i] is set, use basis[j] to
        turn it off
        if ((basis[i] >> j) & 1) {
          basis[i] ^= basis[j];
        }
      }
    }
    // Collect non-zero elements into the reduced_basis array
    (in ascending order)
    for (int i = 0; i < LOG_D; i++) {
      if (basis[i]) {
        reduced_basis.push_back(basis[i]);
      }
    }
  }

  // 6. Get the k-th smallest distinct XOR sum (1-indexed).
  // k = 1 returns 0 (the empty subset).
  // Returns -1 if k is out of bounds (k > 2^sz).
  long long get_kth_smallest(long long k) {
    long long total_combinations = 1LL << sz;
    if (k < 1 || k > total_combinations) return -1;

    long long rank = k - 1; // 0-indexed rank
    long long res = 0;
    for (int i = 0; i < sz; i++) {
      if ((rank >> i) & 1) {
        res ^= reduced_basis[i];
      }
    }
    return res;
  }

  // 7. Get the k-th largest distinct XOR sum (1-indexed).
  // k = 1 returns the absolute maximum possible XOR sum.
  // Returns -1 if k is out of bounds (k > 2^sz).
  long long get_kth_largest(long long k) {
    long long total_combinations = 1LL << sz;
    if (k < 1 || k > total_combinations) return -1;

    // The k-th largest is equivalent to the
    (total_combinations - k)-th smallest
    long long rank = total_combinations - k;
    long long res = 0;
    for (int i = 0; i < sz; i++) {
      if ((rank >> i) & 1) {
        res ^= reduced_basis[i];
      }
    }
    return res;
  }
};
```

## 2.8 Bipartite DSU Rollback

```
/*
   Idea:
   - Supports union of two sets, rolling back to a previous
   state, and
     checking if the current graph is Bipartite (2-colorable).
   - STRICT RULE: NO PATH COMPRESSION! We use "Union by Size"
   only.
   - We maintain `parity[v]`: the color difference between `v`
   and `parent[v]`.
   - We track `bipartite_violations`: the number of odd
   cycles. The graph
     is bipartite if and only if `bipartite_violations == 0`.
*/
struct BipartiteDSU_Rollback {
  vector<int> parent, sz, parity;
  int components;
```

```
int bipartite_violations;

// History structure to record every operation for accurate
rollback
struct Op {
  int child, parent;
  bool added_violation;
};
vector<Op> history;

BipartiteDSU_Rollback(int n) {
  parent.assign(n + 1, 0);
  sz.assign(n + 1, 1);
  parity.assign(n + 1, 0); // 0 = same color as parent, 1 =
  different
  components = n;
  bipartite_violations = 0;

  for (int i = 1; i <= n; i++) {
    parent[i] = i;
  }
}

// Find set WITHOUT path compression.
// Returns {root, color_relative_to_root}
pair<int, int> find_set(int v) {
  int color = 0;
  while (v != parent[v]) {
    color ^= parity[v];
    v = parent[v];
  }
  return {v, color};
}

// Returns true if the entire graph can be 2-colored
bool is_bipartite() {
  return bipartite_violations == 0;
}

// Union two sets and ensure they have DIFFERENT colors
bool union_sets(int a, int b) {
  auto [root_a, color_a] = find_set(a);
  auto [root_b, color_b] = find_set(b);

  // If they are already in the same component
  if (root_a == root_b) {
    if (color_a == color_b) {
      // Same color -> Odd cycle created -> Bipartite
      violated
      bipartite_violations++;
      history.push_back({-1, -1, true});
    } else {
      // Even cycle -> Safe, no structural changes needed
      history.push_back({-1, -1, false});
    }
    return false; // No component merged
  }

  // Union by size
  if (sz[root_a] > sz[root_b]) {
    swap(root_a, root_b);
    swap(color_a, color_b);
  }

  // Perform union
  parent[root_a] = root_b;
  sz[root_b] += sz[root_a];
  components--;

  // Set parity so that 'a' and 'b' have different colors
  // We want: color_a ^ parity[root_a] ^ color_b == 1
  parity[root_a] = color_a ^ color_b ^ 1;

  // Record the operation
  history.push_back({root_a, root_b, false});
  return true;
}

// Take a snapshot of the current state
int snapshot() {
  return history.size();
}

  // Rollback operations to the exact state at `checkpoint`
  void rollback(int checkpoint) {
    while (history.size() > checkpoint) {
      Op op = history.back();
      history.pop_back();

      // Undo bipartite violation if this operation caused one
      if (op.added_violation) {
        bipartite_violations--;
      }

      // Undo structural changes if a union actually happened
      if (op.child != -1) {
        int a = op.child;
        int b = op.parent;

        sz[b] -= sz[a];
        parent[a] = a;
        parity[a] = 0; // Restore parity
        components++;
      }
    }
  }
};
```

## 2.9 Ordered Set

```
/*
     1. find_by_order(k): returns an iterator to the k-th
     smallest element (0-indexed).
     2. order_of_key(x): returns the number of elements
     strictly smaller than x.
*/
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

// 1. Ordered Set (Distinct elements only)
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

// 2. Ordered Multiset (Allows duplicates safely by using
pair<T, int>)
template <typename T>
using ordered_multiset = tree<pair<T, int>, null_type,
less<pair<T, int>>, rb_tree_tag,
tree_order_statistics_node_update>;

/*
   USAGE EXAMPLE:

   signed main() {
     //
     ------------------------------------------------------------
     // EXAMPLE 1: ORDERED SET (Distinct Elements)
     //
     ------------------------------------------------------------
     ordered_set<int> os;
     os.insert(10);
     os.insert(30);
     os.insert(20);
     os.insert(30); // Ignored, as 30 is already present

     // find_by_order(k): Returns iterator to the k-th
     smallest element (0-indexed)
     cout << *os.find_by_order(0) << "\n"; // Output: 10
     cout << *os.find_by_order(2) << "\n"; // Output: 30

     // order_of_key(x): Returns the number of elements
     STRICTLY smaller than x
     cout << os.order_of_key(25) << "\n";  // Output: 2 (10
     and 20)
     cout << os.order_of_key(20) << "\n";  // Output: 1 (only
     10)

     os.erase(20); // Erases 20 normally
```

```
    //
    ------------------------------------------------------------
    // EXAMPLE 2: ORDERED MULTISET (With Duplicates)
    //
    ------------------------------------------------------------
    ordered_multiset<int> oms;
    int timer = 0; // Unique ID counter to distinguish
    duplicates

    // Insert: use pair {value, ++timer}
    oms.insert({10, ++timer});
    oms.insert({20, ++timer});
    oms.insert({20, ++timer});
    oms.insert({30, ++timer});

    // order_of_key: just pass {value, -1} to get count of
    elements < value
    cout << oms.order_of_key({20, -1}) << "\n"; // Output: 1
    (only {10})
    cout << oms.order_of_key({30, -1}) << "\n"; // Output: 3
    ({10}, {20}, {20})

    // find_by_order works exactly the same, but returns a
    pair
    cout << oms.find_by_order(1)->first << "\n"; // Output:
    20
    cout << oms.find_by_order(2)->first << "\n"; // Output:
    20

    // Erase one occurrence of 20:
    // Find its iterator using lower_bound with -1, then
    erase the iterator
    auto it = oms.lower_bound({20, -1});
    if (it != oms.end() && it->first == 20) {
      oms.erase(it);
    }

    return 0;
  }
*/
```

## 2.10 Line Container

```
const long long LINF = 2e18;
struct Line {
  mutable long long k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(long long x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  long long div(long long a, long long b) {
    return a / b - ((a ^ b) < 0 && a % b);
  }

  bool isect(iterator x, iterator y) {
    if (y == end()) { x->p = LINF; return false; }
    if (x->k == y->k) {
      x->p = x->m > y->m ? LINF : -LINF;
    } else {
      x->p = div(y->m - x->m, x->k - y->k);
    }
    return x->p >= y->p;
  }

  // Adds a line y = k*x + m to the envelope
  void add(long long k, long long m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p) {
      isect(x, erase(y));
    }
  }

  // Queries the maximum y value at a given x
  long long query(long long x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};

/*
   USAGE EXAMPLE:
   signed main() {
     LineContainer lc;
     //
     ------------------------------------------------------------
     // EXAMPLE 1: MAXIMUM QUERIES (Default behavior)
     //
     ------------------------------------------------------------
     // Add lines: y = 2x + 3, y = 1x + 10, y = 3x - 5
     lc.add(2, 3);
     lc.add(1, 10);
     lc.add(3, -5);

     cout << "Max at x=2: " << lc.query(2) << "\n"; // y =
     1*(2) + 10 = 12
     cout << "Max at x=10: " << lc.query(10) << "\n"; // y =
     3*(10) - 5 = 25

     //
     ------------------------------------------------------------
     // EXAMPLE 2: MINIMUM QUERIES (Trick)
     //
     ------------------------------------------------------------
     LineContainer min_lc;

     // We want to find MINIMUM of: y = 2x + 3, y = 1x + 10, y
     = 3x - 5
     // Trick: Insert (-k, -m)
     min_lc.add(-2, -3);
     min_lc.add(-1, -10);
     min_lc.add(-3, 5); // -(-5) = 5

     // Trick: Negate the query result
     cout << "Min at x=2: " << -min_lc.query(2) << "\n";  // y
     = 2*(2) + 3 = 7
     cout << "Min at x=10: " << -min_lc.query(10) << "\n"; //
     y = 1*(10) + 10 = 20

     return 0;
   }
*/
```

## 2.11 Deque CHT

```
/*
Trick for MINIMUM QUERIES:
   - Add lines as: add(-k, -m)
   - Query as: -query_increasing_x(x) or
   -query_binary_search(x)
*/
struct Line {
  long long k, m;
  long long get(long long x) { return k * x + m; }
};

struct DequeCHT {
  deque<Line> dq;

  long long floor_div(long long a, long long b) {
    return a / b - ((a ^ b) < 0 && a % b);
  }

  long long intersect(Line a, Line b) {
    return floor_div(a.m - b.m, b.k - a.k);
  }

  // Adds a line y = k*x + m.
  // IMPORTANT: Slopes (k) must be inserted in non-decreasing
  order!
  void add(long long k, long long m) {
    Line L = {k, m};

    if (!dq.empty() && dq.back().k == k) {
      if (m <= dq.back().m) return;
      dq.pop_back();
    }
```

```
    while (dq.size() >= 2) {
      Line l1 = dq[dq.size() - 2];
      Line l2 = dq.back();
      if (intersect(l1, l2) >= intersect(l2, L)) {
        dq.pop_back();
      } else {
        break;
      }
    }
    dq.push_back(L);
  }

  // 1. O(1) Query: ONLY use when query values 'x' are
  monotonically INCREASING!
  long long query_increasing_x(long long x) {
    assert(!dq.empty());
    while (dq.size() >= 2 && dq[0].get(x) <= dq[1].get(x)) {
      dq.pop_front();
    }
    return dq[0].get(x);
  }

  // 2. O(log N) Query: Use when 'x' values are RANDOM / NOT
  SORTED
  long long query_binary_search(long long x) {
    assert(!dq.empty());
    int l = 0, r = dq.size() - 2;
    int ans = dq.size() - 1;

    while (l <= r) {
      int mid = l + (r - l) / 2;
      if (intersect(dq[mid], dq[mid + 1]) < x) {
        l = mid + 1;
      } else {
        ans = mid;
        r = mid - 1;
      }
    }
    return dq[ans].get(x);
  }
};
```

# 3 Graphs

## 3.1 2 SAT

```
/*
  - For a boolean variable x, we create two nodes: x (1..n)
  and NOT(x) (n+1..2n).
  - A clause (x OR y) is equivalent to: (!x => y) and (!y =>
  x).
  - If x and NOT(x) are in the same Strongly Connected
  Component (SCC),
    it's a contradiction -> NO solution.
  - We assign TRUE to x if its SCC is topologically sorted
  AFTER NOT(x)'s SCC.
    (Tarjan assigns SCC IDs in reverse topological order, so
    col[x] < col[NOT(x)])
*/

const int N = 2e4 + 5;

int n;
vector<int> G[2 * N];
int num[2 * N], low[2 * N], cnt;
int col[2 * N], ct;
vector<int> stk;
int ans[N];

int NOT(int x) {
  return (x <= n ? x + n : x - n);
}

void addedge(int x, int y) {
  G[NOT(x)].push_back(y);
  G[NOT(y)].push_back(x);
}

void tarjan(int u) {
  num[u] = low[u] = ++cnt;
  stk.push_back(u);
  for (int v : G[u]) {
    if (!col[v]) {
      if (num[v]) {
        low[u] = min(low[u], num[v]);
      } else {
        tarjan(v);
        low[u] = min(low[u], low[v]);
      }
    }
  }
  if (num[u] == low[u]) {
    ct++;
    while (true) {
      int v = stk.back();
      stk.pop_back();
      col[v] = ct;
      if (v == u) break;
    }
  }
}

bool solve() {
  rep(i, 1, 2 * n) {
    if (!num[i]) tarjan(i);
  }
  rep(i, 1, n) {
    if (col[i] == col[NOT(i)]) return false;
    ans[i] = (col[i] < col[NOT(i)]);
  }
  return true;
}
```

## 3.2 LCA O(1)

```
const int N = 1e5 + 5;
const int LOG = 18;

vector<int> G[N];
int tin[N], cnt, rn[N];
int euler[2 * N], ct, first[N];
int depth[N];
int rmq[LOG][2 * N];

void dfs(int u, int p = 0) {
  tin[u] = ++cnt;
  rn[cnt] = u;
  euler[++ct] = tin[u];
  first[tin[u]] = ct;
  for (int v : G[u]) {
    if (v == p) continue;
    depth[v] = depth[u] + 1;
    dfs(v, u);
    euler[++ct] = tin[u];
  }
}

void build_lca() {
  for (int i = 1; i <= ct; i++) {
    rmq[0][i] = euler[i];
  }
  for (int j = 1; j < LOG; j++) {
    for (int i = 1; i <= ct - (1 << (j - 1)); i++) {
      rmq[j][i] = min(rmq[j - 1][i], rmq[j - 1][i + (1 << (j -
      1))]);
    }
  }
}

int lca(int x, int y) {
  x = first[tin[x]];
  y = first[tin[y]];
  if (x > y) swap(x, y);
  int k = 31 - __builtin_clz(y - x + 1);
  int min_tin = min(rmq[k][x], rmq[k][y - (1 << k) + 1]);
  return rn[min_tin];
}
```

### 3.3 Centroid Decomposition

```
const int N = 3e5 + 5;
vector<int> G[N];
int sz[N];
bool deleted[N];
int centroid_par[N];

void get_sz(int u, int par) {
  sz[u] = 1;
  for (int v : G[u]) {
    if (v != par && !deleted[v]) {
      get_sz(v, u);
      sz[u] += sz[v];
    }
  }
}

int get_centroid(int u, int par, int total_nodes) {
  for (int v : G[u]) {
    if (v != par && !deleted[v]) {
      if (sz[v] > total_nodes / 2) {
        return get_centroid(v, u, total_nodes);
      }
    }
  }
  return u;
}

void solve_centroid(int centroid) {
  // TODO: Add your logic here
}

void build_centroid(int u, int p = -1) {
  get_sz(u, -1);
  int centroid = get_centroid(u, -1, sz[u]);
  deleted[centroid] = true;
  centroid_par[centroid] = p;
  solve_centroid(centroid);
  for (int v : G[centroid]) {
    if (!deleted[v]) {
      build_centroid(v, centroid);
    }
  }
}
```

### 3.4 Heavy Light Decomposition

```
const int N = 2e5 + 5;
vector<int> G[N];
int sz[N], son[N], depth[N], pre[N];
int in[N], cnt, top[N];

void dfs1(int u, int par) {
  sz[u] = 1;
  son[u] = 0;
  pre[u] = par;
  for (int v : G[u]) {
    if (v != par) {
      depth[v] = depth[u] + 1;
      dfs1(v, u);
      sz[u] += sz[v];
      if (sz[v] > sz[son[u]]) {
        son[u] = v;
      }
    }
  }
}

void rebuild(int u, int topp) {
  in[u] = ++cnt;
  top[u] = topp;
  if (son[u]) {
    rebuild(son[u], topp);
  }
  for (int v : G[u]) {
    if (v != pre[u] && v != son[u]) {
      rebuild(v, v);
    }
  }
}

int query_path(int x, int y) {
  int res = 0;
  while (top[x] != top[y]) {
    if (depth[top[x]] < depth[top[y]]) {
      swap(x, y);
    }
    res = max(res, get(1, 1, n, in[top[x]], in[x]));
    x = pre[top[x]];
  }
  if (depth[x] > depth[y]) {
    swap(x, y);
  }
  res = max(res, get(1, 1, n, in[x], in[y]));
  return res;
}

void update_path(int x, int y, int val) {
  while (top[x] != top[y]) {
    if (depth[top[x]] < depth[top[y]]) {
      swap(x, y);
    }
    x = pre[top[x]];
  }
  if (depth[x] > depth[y]) {
    swap(x, y);
  }
}
/*
   USAGE EXAMPLE:
   dfs1(1, 1);
   cnt = 0;
   rebuild(1, 1);
   // for(int i = 1; i <= n; i++) update(1, 1, n, in[i],
   a[i]);
*/
```

### 3.5 Bellman Ford

```
const long long LINF = 1e18;

struct Edge {
  int u, v;
  long long w;
};

int n, m;
vector<Edge> edges;
long long dist[2005];
int parent[2005];

bool bellman_ford(int src) {
  for (int i = 1; i <= n; i++) {
    dist[i] = LINF;
    parent[i] = -1;
  }
  dist[src] = 0;
  for (int i = 1; i <= n - 1; i++) {
    bool updated = false;
    for (auto e : edges) {
      if (dist[e.u] < LINF && dist[e.u] + e.w < dist[e.v]) {
        dist[e.v] = dist[e.u] + e.w;
        parent[e.v] = e.u;
        updated = true;
      }
    }
    if (!updated) break;
  }
  bool has_negative_cycle = false;
  for (int i = 1; i <= n; i++) {
    for (auto e : edges) {
      if (dist[e.u] < LINF && dist[e.u] + e.w < dist[e.v]) {
        dist[e.v] = -LINF;
        has_negative_cycle = true;
      }
    }
  }
  return has_negative_cycle;
}

/*
```

```
   HOW TO EXTRACT A NEGATIVE CYCLE (If needed)
   If you only need to print ONE negative cycle (like CSES
   Cycle Finding):
   1. Keep an array `parent`.
   2. Run Bellman-Ford for exactly N iterations (no need for
   Phase 2 above).
   3. On the N-th iteration, if an edge (u, v) is relaxed, `v`
   is either in
      the negative cycle or reachable from it.
   4. To find a node strictly inside the cycle, go back `N`
   times using `parent`:
      int x = v;
      for (int i = 1; i <= n; i++) x = parent[x];
   5. Now `x` is guaranteed to be in the cycle. Trace back
   using `parent[x]`
      until you reach `x` again, push nodes to a vector, and
      reverse it.
*/
```

### 3.6 Dinic

```
/*
   Complexity:
   - General Graph: O(V^2 * E)
   - Unit Network (e.g., Bipartite Matching, weight = 1): O(E
   * sqrt(V))
   - Extremely fast in practice, much faster than
   Edmonds-Karp.
*/
const long long LINF = 1e18;

struct Edge {
  int u, v;
  long long cap, flow;
};

struct Dinic {
  int n;
  vector<Edge> edges;
  vector<vector<int>> adj;
  vector<int> level;
  vector<int> ptr;

  Dinic(int _n) {
    n = _n;
    adj.assign(n + 1, vector<int>());
    level.assign(n + 1, -1);
    ptr.assign(n + 1, 0);
  }

  void add_edge(int u, int v, long long cap, bool is_directed
  = true) {
    adj[u].push_back(edges.size());
    edges.push_back({u, v, cap, 0});

    adj[v].push_back(edges.size());
    edges.push_back({v, u, is_directed ? 0 : cap, 0});
  }

  bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    level[s] = 0;
    queue<int> q;
    q.push(s);

    while (!q.empty()) {
      int u = q.front();
      q.pop();
      for (int id : adj[u]) {
        int v = edges[id].v;
        if (level[v] == -1 && edges[id].cap - edges[id].flow >
        0) {
          level[v] = level[u] + 1;
          q.push(v);
        }
      }
    }
    return level[t] != -1;
  }

  long long dfs(int u, int t, long long pushed) {
    if (pushed == 0 || u == t) return pushed;

    for (int& cid = ptr[u]; cid < adj[u].size(); ++cid) {
      int id = adj[u][cid];
      int v = edges[id].v;
      long long tr = edges[id].cap - edges[id].flow;

      if (level[u] + 1 != level[v] || tr == 0) continue;

      long long push = dfs(v, t, min(pushed, tr));
      if (push == 0) continue;

      edges[id].flow += push;
      edges[id ^ 1].flow -= push;
      return push;
    }
    return 0;
  }

  // Main function to calculate Max Flow
  long long max_flow(int s, int t) {
    long long flow = 0;
    while (bfs(s, t)) {
      fill(ptr.begin(), ptr.end(), 0);

      while (long long pushed = dfs(s, t, LINF)) {
        flow += pushed;
      }
    }
    return flow;
  }

  // Min Cut Extraction: Nodes reachable from Source in the
  residual graph
  vector<bool> get_min_cut(int s) {
    vector<bool> visited(n + 1, false);
    queue<int> q;
    q.push(s);
    visited[s] = true;

    while (!q.empty()) {
      int u = q.front();
      q.pop();
      for (int id : adj[u]) {
        int v = edges[id].v;
        if (!visited[v] && edges[id].cap - edges[id].flow > 0)
        {
          visited[v] = true;
          q.push(v);
        }
      }
    }
    return visited; // visited[i] = true means 'i' is in the
    Source set
  }

  // Tracing: Print edges that carry flow
  void print_flow_paths() {
    cout << "Edges with positive flow:\n";
    for (int i = 0; i < edges.size(); i += 2) { // Check
    forward edges only
      if (edges[i].flow > 0) {
        cout << edges[i].u << " -> " << edges[i].v
             << " : " << edges[i].flow << " / " <<
             edges[i].cap << "\n";
      }
    }
  }
};
/*
   USAGE EXAMPLE:
   signed main() {
     int n = 4; // 4 vertices
     Dinic dinic(n);
     // add_edge(u, v, capacity)
     dinic.add_edge(1, 2, 3);
     dinic.add_edge(1, 3, 2);
     dinic.add_edge(2, 3, 1);
     dinic.add_edge(2, 4, 2);
     dinic.add_edge(3, 4, 4);
     int source = 1, sink = 4;
```

```
    cout << "Max Flow: " << dinic.max_flow(source, sink) <<
    "\n"; // Output: 5
    return 0;
  }
*/
```

### 3.7 Flow with lower bound

```
/*
  REQUIREMENT:
  - Paste the Dinic template (Edge + Dinic + LINF) BEFORE
  this template.
  - Original vertices are numbered from 1 to n.
  - Each edge has a lower bound l and upper bound r: l <=
  flow <= r.
  - Each LowerBoundFlow object supports ONE solve call only.
  - max_flow/min_flow consider non-negative s -> t flow
  values.

  Complexity:
  - General Graph: O(V^2 * E), up to 2 Max Flow runs.
*/

struct LowerBoundFlow {
  struct OriginalEdge {
    int u, v, id;
    ll lower, upper;
  };

  int n, ss, tt;
  Dinic dinic;
  vector<ll> demand;
  vector<OriginalEdge> original_edges;
  bool used = false, feasible = false;
  ll base_flow = 0;

  LowerBoundFlow(int _n) : n(_n), ss(n + 1), tt(n + 2),
  dinic(n + 2) {
    demand.assign(n + 3, 0);
  }

  // Add edge u -> v with l <= flow <= r.
  // Return an ID (0-based) for get_flow().
  int add_edge(int u, int v, ll l, ll r) {
    assert(!used && 1 <= u && u <= n && 1 <= v && v <= n);
    assert(0 <= l && l <= r);

    int id = dinic.edges.size();
    dinic.add_edge(u, v, r - l);

    demand[u] -= l;
    demand[v] += l;

    original_edges.push_back({u, v, id, l, r});
    return (int)original_edges.size() - 1;
  }

  // Actual flow on an original edge, INCLUDING its lower
  bound.
  // Call only after a successful solve.
  ll get_flow(int edge_id) const {
    assert(feasible && 0 <= edge_id && edge_id <
    (int)original_edges.size());
    const auto &e = original_edges[edge_id];
    return e.lower + dinic.edges[e.id].flow;
  }

  // Disable both directions of an auxiliary edge in the
  residual graph.
  void disable_edge(int id) {
    dinic.edges[id].cap = dinic.edges[id].flow = 0;
    dinic.edges[id ^ 1].cap = dinic.edges[id ^ 1].flow = 0;
  }

  // Internal: build and check feasible circulation.
  // If s,t are supplied, temporarily add t -> s with capacity
  LINF.
  bool prepare(int s = -1, int t = -1) {
    assert(!used);
    used = true;
    int back_edge = -1;
    if (s != -1) {
      assert(1 <= s && s <= n && 1 <= t && t <= n && s != t);
      back_edge = dinic.edges.size();
      dinic.add_edge(t, s, LINF);
    }

    vector<int> auxiliary_edges;
    ll required = 0;

    for (int v = 1; v <= n; v++) {
      if (demand[v] > 0) {
        int id = dinic.edges.size();
        dinic.add_edge(ss, v, demand[v]);
        auxiliary_edges.push_back(id);
        required += demand[v];
      } else if (demand[v] < 0) {
        int id = dinic.edges.size();
        dinic.add_edge(v, tt, -demand[v]);
        auxiliary_edges.push_back(id);
      }
    }

    if (dinic.max_flow(ss, tt) != required) return false;

    feasible = true;
    if (back_edge != -1) {
      base_flow = dinic.edges[back_edge].flow;
      disable_edge(back_edge);
    }

    for (int id : auxiliary_edges) disable_edge(id);
    return true;
  }

  // Max flow with a limit, using the EXISTING Dinic bfs() and
  dfs().
  ll max_flow_limited(int s, int t, ll limit) {
    ll ans = 0;
    while (ans < limit && dinic.bfs(s, t)) {
      fill(dinic.ptr.begin(), dinic.ptr.end(), 0);
      while (ans < limit) {
        ll pushed = dinic.dfs(s, t, min(LINF, limit - ans));
        if (!pushed) break;
        ans += pushed;
      }
    }
    return ans;
  }

  // No source/sink: conservation at ALL vertices.
  bool feasible_circulation() {
    return prepare();
  }

  // Check existence of a non-negative s -> t flow.
  bool feasible_flow(int s, int t) {
    return prepare(s, t);
  }

  // Return {exists, maximum flow value}.
  pair<bool, ll> max_flow(int s, int t) {
    if (!prepare(s, t)) return {false, 0};
    return {true, base_flow + dinic.max_flow(s, t)};
  }

  // Return {exists, minimum NON-NEGATIVE flow value}.
  pair<bool, ll> min_flow(int s, int t) {
    if (!prepare(s, t)) return {false, 0};
    return {true, base_flow - max_flow_limited(t, s,
    base_flow)};
  }

  // Tracing: print all original edges carrying positive flow.
  void print_flow_paths() const {
    assert(feasible);
    cout << "Edges with positive flow:\n";
    for (int i = 0; i < (int)original_edges.size(); i++) {
      ll f = get_flow(i);
      if (f > 0) {
        const auto &e = original_edges[i];
        cout << e.u << " -> " << e.v << " : "
```

```
            << f << " / [" << e.lower << ", " << e.upper <<
            "]\n";
      }
    }
  }
};
/*
   USAGE EXAMPLES:

   // 1. Maximum s -> t flow:
   LowerBoundFlow f(n);
   int id = f.add_edge(1, 2, 2, 5);
   f.add_edge(2, 3, 1, 7);

   auto [ok, ans] = f.max_flow(1, 3);
   if (!ok) cout << -1 << '\n';
   else {
     cout << "Max Flow: " << ans << '\n';
     cout << "Flow on edge id: " << f.get_flow(id) << '\n';
     f.print_flow_paths();
   }

   // 2. Minimum NON-NEGATIVE s -> t flow (use a NEW object):
   LowerBoundFlow g(n);
   // g.add_edge(u, v, l, r);
   auto [ok2, min_ans] = g.min_flow(s, t);

   // 3. Feasible circulation (no designated source/sink):
   LowerBoundFlow h(n);
   // h.add_edge(u, v, l, r);
   bool possible = h.feasible_circulation();

   // 4. Just check whether a non-negative s -> t flow exists:
   LowerBoundFlow k(n);
   // k.add_edge(u, v, l, r);
   bool possible_st = k.feasible_flow(s, t);

   NOTES:
   - Do NOT call two solve methods on the same object.
   - Do NOT add t -> s yourself when using
   max_flow/min_flow/feasible_flow.
   - get_flow(id) is available only after a successful solve.
*/
```

### 3.8 Min Cost Max Flow

```
/*
   - Uses Dijkstra instead of SPFA to find the cheapest
   augmenting path.
   - Maintains a potential function `pi` for each node to
   handle negative
     weight residual edges.
   - Edge weights are reweighted: w'(u, v) = w(u, v) + pi[u] -
   pi[v] >= 0.
   - Complexity: O(F * E log V) where F is the flow. Fast and
   safe!
*/
const long long LINF = 1e18;

struct MCMFEdge {
  int u, v;
  long long cap, flow, cost;
};

struct MinCostMaxFlow {
  int n;
  vector<MCMFEdge> edges;
  vector<vector<int>> adj;
  vector<long long> pi, dist;
  vector<int> parent_edge;

  MinCostMaxFlow(int _n) {
    n = _n;
    adj.assign(n + 1, vector<int>());
    pi.assign(n + 1, 0);
    dist.assign(n + 1, 0);
    parent_edge.assign(n + 1, -1);
  }

  void add_edge(int u, int v, long long cap, long long cost,
  bool is_directed = true) {
    adj[u].push_back(edges.size());
    edges.push_back({u, v, cap, 0, cost});

    adj[v].push_back(edges.size());
    edges.push_back({v, u, is_directed ? 0 : cap, 0, -cost});
  }

  // Run SPFA once to set initial potentials.
  // ONLY UNCOMMENT inside mcmf() if the original graph has
  NEGATIVE edge costs.
  void set_pi(int s) {
    fill(pi.begin(), pi.end(), LINF);
    vector<bool> in_queue(n + 1, false);
    queue<int> q;

    pi[s] = 0;
    q.push(s);
    in_queue[s] = true;

    while (!q.empty()) {
      int u = q.front();
      q.pop();
      in_queue[u] = false;

      for (int id : adj[u]) {
        int v = edges[id].v;
        if (edges[id].cap - edges[id].flow > 0 && pi[v] >
        pi[u] + edges[id].cost) {
          pi[v] = pi[u] + edges[id].cost;
          if (!in_queue[v]) {
            q.push(v);
            in_queue[v] = true;
          }
        }
      }
    }
  }

  // Dijkstra using reduced costs
  bool dijkstra(int s, int t) {
    fill(dist.begin(), dist.end(), LINF);
    fill(parent_edge.begin(), parent_edge.end(), -1);

    priority_queue<pair<long long, int>, vector<pair<long
    long, int>>, greater<pair<long long, int>>> pq;

    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
      long long d = pq.top().first;
      int u = pq.top().second;
      pq.pop();

      if (d > dist[u]) continue;

      for (int id : adj[u]) {
        int v = edges[id].v;
        long long remain_cap = edges[id].cap - edges[id].flow;

        if (remain_cap > 0) {
          long long reduced_cost = edges[id].cost + pi[u] -
          pi[v];
          if (dist[v] > dist[u] + reduced_cost) {
            dist[v] = dist[u] + reduced_cost;
            parent_edge[v] = id;
            pq.push({dist[v], v});
          }
        }
      }
    }
    return dist[t] != LINF;
  }

  pair<long long, long long> mcmf(int s, int t, long long
  max_required_flow = LINF) {
    long long total_flow = 0;
    long long total_cost = 0;

    // Uncomment this line ONLY IF the initial graph contains
    negative costs
```

```
    // set_pi(s);

    while (total_flow < max_required_flow && dijkstra(s, t)) {
      long long push = max_required_flow - total_flow;
      int cur = t;
      while (cur != s) {
        int id = parent_edge[cur];
        push = min(push, edges[id].cap - edges[id].flow);
        cur = edges[id].u;
      }

      cur = t;
      while (cur != s) {
        int id = parent_edge[cur];
        edges[id].flow += push;
        edges[id ^ 1].flow -= push;
        total_cost += push * edges[id].cost;
        cur = edges[id].u;
      }

      total_flow += push;

      for (int i = 1; i <= n; i++) {
        if (dist[i] != LINF) {
          pi[i] += dist[i];
        }
      }
    }

    return {total_flow, total_cost};
  }

  // PATH TRACING: Prints all edges that are actively carrying
  flow
  void print_flow_paths() {
    cout << "Edges with positive flow (Min Cost Max Flow):\n";
    for (int i = 0; i < edges.size(); i += 2) {
      if (edges[i].flow > 0) {
        cout << edges[i].u << " -> " << edges[i].v
             << " | Flow: " << edges[i].flow << " / " <<
             edges[i].cap
             << " | Unit Cost: " << edges[i].cost
             << " | Total Edge Cost: " << edges[i].flow *
             edges[i].cost << "\n";
      }
    }
  }
};
```

## 3.9 Dsu On Tree(Sack)

```
const int N = 2e5 + 5;
vector<int> G[N];

int in[N], out[N], rn[N], timer;
int sz[N], big[N];

// 1. First DFS: Precompute subtree sizes, Euler tour, and
heavy children
void dfs_sz(int u, int par) {
  in[u] = ++timer;
  rn[timer] = u;
  sz[u] = 1;
  big[u] = 0;

  for (int v : G[u]) {
    if (v != par) {
      dfs_sz(v, u);
      sz[u] += sz[v];
      if (sz[big[u]] < sz[v]) {
        big[u] = v; // Update heavy child
      }
    }
  }
  out[u] = timer;
}

// Struct to manage the global state (Modify this according to
the problem!)
struct GlobalState {
  int freq[N];
  int count_of_freq[N];

  void add(int node) {
  }

  void remove(int node) {
  }

} state;


// 2. Second DFS: DSU on Tree (Sack)
void dsu_on_tree(int u, int par, bool keep) {
  for (int v : G[u]) {
    if (v != par && v != big[u]) {
      dsu_on_tree(v, u, false);
    }
  }

  if (big[u] != 0) {
    dsu_on_tree(big[u], u, true);
  }

  for (int v : G[u]) {
    if (v != par && v != big[u]) {
      for (int i = in[v]; i <= out[v]; i++) {
        state.add(rn[i]);
      }
    }
  }
  state.add(u);

  // ANSWER QUERIES for node `u` here

  if (!keep) {
    for (int i = in[u]; i <= out[u]; i++) {
      state.remove(rn[i]);
    }
  }
}
```

## 3.10 MO On Tree

```
/*
   MO'S ALGORITHM ON TREES (WITH UPDATES - 3D MO)
   Idea:
   - Flattens the tree into an array using Euler Tour. Each
   node appears
     exactly TWICE: when we enter it (tin) and when we leave
     it (tout).
   - A node is considered "in the current path" if it appears
   exactly ONCE
     in our active range [L, R]. We maintain a boolean array
     `f[u]` to
     toggle the presence of node `u`.

   Query Translation (Path from U to V, assuming tin[U] <
   tin[V]):
   - Case 1: U is an ancestor of V.
             Query range: [tin[U], tin[V]]. LCA is already
             included.
   - Case 2: U is NOT an ancestor of V.
             Query range: [tout[U], tin[V]]. LCA is NOT in
             this range,
             so we must manually add/remove the LCA during the
             query.
   Complexity:
   - Without updates (2D Mo): Block size = N / sqrt(Q), Time =
   O(N * sqrt(Q))
   - With updates (3D Mo): Block size = N^(2/3), Time =
   O(N^(5/3))
*/
const int N = 2e5 + 5;
// For 3D Mo (with updates), BLOCK size should be roughly
N^(2/3).
// For N = 100,000, N^(2/3) is approx 2150.
const int BLOCK = 2150;

int n, q;
int a[N];
vector<int> G[N];
```

```cpp
// Euler Tour & LCA
int tin[N], tout[N], timer;
int tour[N * 2]; // Needs to be 2 * N because each node
appears twice
int up[18][N], depth[N];

// Mo's State
int freq[N], distinct_count;
int f[N]; // f[x] = 1 if node x is currently in the path, 0
otherwise
int ans[N];

struct Query {
  int l, r, lca_idx, t, id;

  bool operator<(const Query& other) const {
    int bl = l / BLOCK;
    int br = r / BLOCK;
    int obl = other.l / BLOCK;
    int obr = other.r / BLOCK;
    if (bl != obl) return bl < obl;
    if (br != obr) return (bl & 1) ? br < obr : br > obr;
    return (br & 1) ? t < other.t : t > other.t;
  }
};

struct Update {
  int x, new_val, old_val;
};

vector<Query> queries;
vector<Update> updates;

// 1. Build Euler Tour and LCA table
void dfs(int u, int par) {
  up[0][u] = par;
  for (int i = 1; i < 18; i++) {
    if (up[i - 1][u] != -1) {
      up[i][u] = up[i - 1][up[i - 1][u]];
    } else {
      up[i][u] = -1;
    }
  }

  tin[u] = ++timer;
  tour[timer] = u;

  for (int v : G[u]) {
    if (v != par) {
      depth[v] = depth[u] + 1;
      dfs(v, u);
    }
  }

  tout[u] = ++timer;
  tour[timer] = u;
}

int get_lca(int u, int v) {
  if (depth[u] < depth[v]) swap(u, v);
  int delta = depth[u] - depth[v];
  for (int i = 0; i < 18; i++) {
    if ((delta >> i) & 1) u = up[i][u];
  }
  if (u == v) return u;
  for (int i = 17; i >= 0; i--) {
    if (up[i][u] != up[i][v]) {
      u = up[i][u];
      v = up[i][v];
    }
  }
  return up[0][u];
}

// 2. State Manipulation Functions
void add(int val) {
  if (++freq[val] == 1) distinct_count++;
}

void sub(int val) {
  if (freq[val]-- == 1) distinct_count--;
}

// Toggle a node: if it's already in the path, remove it.
Otherwise, add it.
void toggle(int idx) {
  int x = tour[idx];
  f[x] ^= 1;
  if (f[x]) add(a[x]);
  else sub(a[x]);
}

// Apply point update dynamically
void apply_update(int x, int val, int old_val) {
  if (f[x]) {
    add(val);
    sub(old_val);
  }
  a[x] = val;
}
```

## 3.11 Dynamic Connectivity

```cpp
struct DSU_Rollback {
  vector<int> parent, sz;
  vector<pair<int, int>> history;

  DSU_Rollback(int n) {
    parent.assign(n + 1, 0);
    sz.assign(n + 1, 1);
    for (int i = 1; i <= n; i++) parent[i] = i;
  }

  int find_set(int v) {
    while (v != parent[v]) v = parent[v];
    return v;
  }

  bool union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a == b) return false;

    if (sz[a] > sz[b]) swap(a, b);
    parent[a] = b;
    sz[b] += sz[a];
    history.push_back({a, b});
    return true;
  }

  int snapshot() {
    return history.size();
  }

  void rollback(int checkpoint) {
    while (history.size() > checkpoint) {
      int a = history.back().first;
      int b = history.back().second;
      history.pop_back();

      sz[b] -= sz[a];
      parent[a] = a;
    }
  }
};

struct DynamicConnectivity {
  int max_time;
  DSU_Rollback dsu;
  vector<vector<pair<int, int>>> tree;
  vector<vector<pair<int, int>>> queries; // queries[time] =
  list of {u, v}
  vector<int> ans; // stores answer (1 or 0) for each query
  sequentially

  DynamicConnectivity(int n, int _max_time) :
  max_time(_max_time), dsu(n) {
    tree.assign(4 * max_time + 1, vector<pair<int, int>>());
    queries.assign(max_time + 1, vector<pair<int, int>>());
  }
```

```
  void add_edge_interval(int node, int l, int r, int ql, int
  qr, pair<int, int> edge) {
    if (ql > r || qr < l) return;
    if (ql <= l && r <= qr) {
      tree[node].push_back(edge);
      return;
    }
    int mid = l + (r - l) / 2;
    add_edge_interval(2 * node, l, mid, ql, qr, edge);
    add_edge_interval(2 * node + 1, mid + 1, r, ql, qr, edge);
  }

  // Add an edge that exists from start_time to end_time
  void add_edge(int u, int v, int start_time, int end_time) {
    if (start_time > end_time) return;
    add_edge_interval(1, 1, max_time, start_time, end_time,
    {u, v});
  }

  // Register a connectivity query at a specific time
  void add_query(int u, int v, int time) {
    queries[time].push_back({u, v});
  }

  // Traverse the time tree and answer queries
  void dfs(int node, int l, int r) {
    int checkpoint = dsu.snapshot();

    // Apply all edges covering this node
    for (auto edge : tree[node]) {
      dsu.union_sets(edge.first, edge.second);
    }

    if (l == r) {
      // Leaf node: Answer queries exactly at time 'l'
      for (auto q : queries[l]) {
        ans.push_back(dsu.find_set(q.first) ==
        dsu.find_set(q.second));
      }
    } else {
      // Internal node: Recurse left and right
      int mid = l + (r - l) / 2;
      dfs(2 * node, l, mid);
      dfs(2 * node + 1, mid + 1, r);
    }

    // Rollback the DSU state before going back up
    dsu.rollback(checkpoint);
  }

  // Solves and returns a vector containing answers to all
  queries
  vector<int> solve() {
    if (max_time >= 1) {
      dfs(1, 1, max_time);
    }
    return ans;
  }
};

/*
   USAGE EXAMPLE (Handling Map for Edge Lifespans):

   signed main() {
     int n = 5;  // 5 nodes
     int q = 6;  // 6 events/queries

     DynamicConnectivity dc(n, q);
     map<pair<int, int>, int> edge_start;

     // Note: Always normalize edges so u < v to avoid {u, v}
     and {v, u} mix-ups.
     auto normalize = [](int u, int v) {
       return u < v ? make_pair(u, v) : make_pair(v, u);
     };

     // Simulating reading queries:
     // Time 1: Add edge (1, 2)
     edge_start[normalize(1, 2)] = 1;

     // Time 2: Add edge (2, 3)
     edge_start[normalize(2, 3)] = 2;

     // Time 3: Query (1, 3) -> should be connected
     dc.add_query(1, 3, 3);

     // Time 4: Remove edge (1, 2)
     pair<int, int> e = normalize(1, 2);
     dc.add_edge(e.first, e.second, edge_start[e], 4 - 1);
     edge_start.erase(e); // Edge doesn't exist after time 3

     // Time 5: Query (1, 3) -> should NOT be connected
     dc.add_query(1, 3, 5);

     // Time 6: Add edge (1, 3)
     edge_start[normalize(1, 3)] = 6;

     // IMPORTANT: Close any edges that were never removed!
     for (auto it : edge_start) {
       dc.add_edge(it.first.first, it.first.second, it.second,
       q);
     }

     // Solve and print
     vector<int> results = dc.solve();
     for (int res : results) {
       cout << (res ? "YES" : "NO") << "\n";
     }

     // Expected Output:
     // YES (at time 3)
     // NO (at time 5)

     return 0;
   }
*/
```

# 4 DP

## 4.1 DP Digit Optimization 1

```
/*
  DIGIT DP OPTIMIZATION 1: REMOVING THE `is_less` (fr)
  DIMENSION
*/

vector<int> R;
pair<ll, ll> dp[20];

pair<ll, ll> calc(int pos, int fr) {
  if (pos == -1) return make_pair(0, 1);
  auto &res = dp[pos];
  if (fr && res != make_pair(-1LL, -1LL)) return res;
  int lim_r = (fr ? 9 : R[pos]);
  pair<ll, ll> val = make_pair(0, 0);
  for (int i = 0; i <= lim_r; i++) {
    pair<ll, ll> nxt = calc(pos - 1, fr | (i < lim_r));
    val.first += nxt.first + i * nxt.second;
    val.second += nxt.second;
  }
  return (fr ? res = val : val);
}
```

## 4.2 DP Digit Optimization 2

```
/*
  DIGIT DP OPTIMIZATION 2: ONE-TIME MEMSET FOR MULTI-TEST
  CASES
*/

const int mod = 1e9 + 7;

string R;
int dp[51][51][51][51];

int calc(int pos, int c3, int c6, int c9, int fr) {
  if (pos == -1) {
    return (c3 > 0 && c3 == c6 && c6 == c9);
  }
  auto &res = dp[pos][c3][c6][c9];
  if (fr && res != -1) return res;
  ll val = 0;
```

```
  int lim_r = (fr ? 9 : R[pos] - '0');
  for (int i = 0; i <= lim_r; i++) {
    val += calc(pos - 1, c3 + (i == 3), c6 + (i == 6), c9 + (i
    == 9), fr | (i < lim_r));
    if (val >= mod) val -= mod;
  }
  return (fr ? res = val : val);
}

int solve(string x) {
  reverse(x.begin(), x.end());
  R = x;
  return calc((int)R.size() - 1, 0, 0, 0, 0);
}

bool check(string x) {
  int c3 = 0, c6 = 0, c9 = 0;
  for (auto i : x) {
    c3 += (i == '3');
    c6 += (i == '6');
    c9 += (i == '9');
  }
  return (c3 > 0 && c3 == c6 && c6 == c9);
}

/*
// Usage in main():
// memset(dp, -1, sizeof dp); // Call this ONLY ONCE
// while (test--) {
//   string a, b; cin >> a >> b;
//   cout << (1LL * solve(b) - solve(a) + check(a) + mod) %
mod << "\n";
// }
*/
```

## 4.3 DP SoS

```
const int LOG = 20;
int dp[1 << LOG];

// 1. Sum over Submasks: dp[mask] = \sum_{sub \subseteq mask}
A[sub]
// (sub is formed by turning OFF some bits of mask)
void sos_submask(int n, int a[]) {
  for (int mask = 0; mask < (1 << n); mask++) {
    dp[mask] = a[mask];
  }
  for (int i = 0; i < n; i++) {
    for (int mask = 0; mask < (1 << n); mask++) {
      if ((mask >> i) & 1) {
        dp[mask] += dp[mask ^ (1 << i)];
      }
    }
  }
}

// 2. Sum over Supermasks: dp[mask] = \sum_{mask \subseteq
sup} A[sup]
// (sup is formed by turning ON some bits of mask)
void sos_supermask(int n, int a[]) {
  for (int mask = 0; mask < (1 << n); mask++) {
    dp[mask] = a[mask];
  }
  for (int i = 0; i < n; i++) {
    for (int mask = 0; mask < (1 << n); mask++) {
      if (!((mask >> i) & 1)) {
        dp[mask] += dp[mask ^ (1 << i)];
      }
    }
  }
}
/*
  COMMON USAGE PATTERNS & TRICKS:
   1. Find pairs (u, v) such that u & v = 0:
   2. Number of elements `x` such that (x & mask) == x:
   3. Number of elements `x` such that (x & mask) == mask:
   4. Finding two elements x, y with maximum (x | y):
*/
```

## 4.4 DP Divide And Conquer

```
/*
   DIVIDE AND CONQUER DP OPTIMIZATION

   Idea:
   - Optimizes DP equations of the form:
     dp[i][j] = min_{k < j} (dp[i-1][k] + cost(k+1, j))
   - Condition: The optimal split point `opt(i, j)` must be
   monotonic,
     meaning: opt(i, j) <= opt(i, j+1).
   - This monotonicity allows us to use Divide & Conquer: if
   we know the
     optimal split point for mid is `opt_mid`, then the split
     point for
     any j < mid is <= opt_mid, and for any j > mid is >=
     opt_mid.

   Complexity:
   - Original DP: O(K * N^2)
   - D&C Optimized: O(K * N log N)

   Memory Optimization:
   - We only need the DP state from the previous phase (i -
   1), so we can
     use two 1D arrays: `dp_before` and `dp_current`.
*/
const long long LINF = 1e18;

vector<long long> dp_before, dp_current;

// 1. Problem-specific cost function
// Note: To keep the overall complexity O(K * N log N), this
cost function
// MUST be calculable in O(1). Usually done via prefix sums or
sliding window.
long long cost(int l, int r) {
  if (l > r) return 0;
  // TODO: Replace with the actual O(1) cost logic of the
  problem
  return 0;
}

// 2. Divide and Conquer recursive function
// l, r: the range of `j` we are currently computing
dp_current[j] for.
// optl, optr: the valid range of the optimal split point `k`
for this range.
void compute(int l, int r, int optl, int optr) {
  if (l > r) return;
  int mid = l + (r - l) / 2;
  pair<long long, int> best = {LINF, -1};
  for (int k = optl; k <= min(mid - 1, optr); k++) {
    if (dp_before[k] != LINF) {
      long long current_val = dp_before[k] + cost(k + 1, mid);
      if (current_val < best.first) {
        best = {current_val, k};
      }
    }
  }
  dp_current[mid] = best.first;
  int opt = best.second;
  compute(l, mid - 1, optl, opt);
  compute(mid + 1, r, opt, optr);
}
```

# 5 Strings

## 5.1 KMP

```
vector<int> compute_pi(const string& p) {
  int m = p.length();
  vector<int> pi(m, 0);
  for (int i = 1, j = 0; i < m; i++) {
    while (j > 0 && p[i] != p[j]) {
      j = pi[j - 1];
    }
    if (p[i] == p[j]) {
      j++;
    }
    pi[i] = j;
```

```
  }
  return pi;
}
```

## 5.2 Z Function

```
vector<int> z_function(const string& s) {
  int n = (int)s.length();
  vector<int> z(n, 0);
  for (int i = 1, l = 0, r = 0; i < n; i++) {
    if (i <= r) {
      z[i] = min(r - i + 1, z[i - l]);
    }
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
      z[i]++;
    }
    if (i + z[i] - 1 > r) {
      l = i;
      r = i + z[i] - 1;
    }
  }
  return z;
}
```

## 5.3 Suffix Array

```
/*
   SUFFIX ARRAY & LCP ARRAY (O(N log N) Construction)
   Idea:
   - Suffix Array (sa): Sorts all suffixes of a string
   lexicographically.
     sa[i] stores the starting index of the i-th
     lexicographically
     smallest suffix.
   - LCP Array (lcp): Stores the Longest Common Prefix between
   consecutive
     suffixes in the sorted Suffix Array. lcp[i] = LCP(sa[i],
     sa[i-1]).
   Algorithm:
   - Appends a special character '$' (strictly smaller than
   all other characters)
     to the end of the string.
   - Sorts substrings of length 2^k using Radix Sort in O(N).
   - Repeats log(N) times -> Total time O(N log N).
   - Computes LCP in O(N) using Kasai's Algorithm.
*/

struct SuffixArray {
  int n;
  string s;
  vector<int> sa;  // Suffix Array
  vector<int> c;   // Equivalence classes
  vector<int> lcp; // Longest Common Prefix array

  // Constructor automatically builds SA and LCP
  SuffixArray(string const& _s) : s(_s + "$") {
    n = s.length();
    sa.assign(n, 0);
    c.assign(n, 0);
    lcp.assign(n, 0);

    build_sa();
    build_lcp();
  }

  void build_sa() {
    const int alphabet = 256;
    vector<int> cnt(max(alphabet, n), 0);
    vector<int> p(n), pn(n), cn(n);

    // 1. Initial sort for length 1 (k = 0)
    for (int i = 0; i < n; i++) cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
    for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;

    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
      if (s[p[i]] != s[p[i - 1]]) classes++;
      c[p[i]] = classes - 1;
    }

    // 2. Transitions for length 2^h
    for (int h = 0; (1 << h) < n; h++) {
      // Shift by 2^h
      for (int i = 0; i < n; i++) {
        pn[i] = p[i] - (1 << h);
        if (pn[i] < 0) pn[i] += n;
      }

      // Radix Sort on the first half
      fill(cnt.begin(), cnt.begin() + classes, 0);
      for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
      for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
      for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] =
      pn[i];

      // Update equivalence classes
      cn[p[0]] = 0;
      classes = 1;
      for (int i = 1; i < n; i++) {
        pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) %
        n]};
        pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + (1
        << h)) % n]};
        if (cur != prev) classes++;
        cn[p[i]] = classes - 1;
      }
      c.swap(cn);
    }
    sa = p;
  }

  // Kasai's algorithm to build LCP array in O(N)
  void build_lcp() {
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++) {
      rank[sa[i]] = i;
    }

    int k = 0;
    // Iterate through original string indices (excluding the
    appended '$')
    for (int i = 0; i < n - 1; i++) {
      int pi = rank[i];
      int j = sa[pi - 1]; // The suffix preceding the current
      one in SA

      // Expand the matching prefix
      while (i + k < n && j + k < n && s[i + k] == s[j + k]) {
        k++;
      }

      lcp[pi] = k;
      if (k > 0) k--; // Move to the next suffix, LCP
      decreases by at most 1
    }
  }
};

/*
   USAGE EXAMPLES:

   signed main() {
     string str = "ababba";
     SuffixArray sa(str);

     // 1. Number of Distinct Substrings
     // Formula: Total substrings - sum of LCP array
     // Total substrings of length N is N * (N + 1) / 2
     long long N = str.length();
     long long distinct_substrings = N * (N + 1) / 2;
     for (int i = 1; i <= N; i++) {
       distinct_substrings -= sa.lcp[i];
     }
     cout << "Distinct substrings: " << distinct_substrings <<
     "\n";

     // 2. Printing Suffix Array & LCP Array
     cout << "i\tSA[i]\tLCP[i]\tSuffix\n";
     // Note: sa.sa[0] is always the appended '$' (length 0
     suffix)
```

```
    for (int i = 1; i <= N; i++) {
      cout << i << "\t"
           << sa.sa[i] << "\t"
           << sa.lcp[i] << "\t"
           << str.substr(sa.sa[i]) << "\n";
    }

    return 0;
  }
*/
```

## 5.4 Aho Corasick

```
/*
   AHO-CORASICK AUTOMATON
   Idea:
   - Multi-pattern string search algorithm.
   - Constructs a Trie from a set of patterns and adds:
     1. Failure Links (`fail`): Where to go when a character
     mismatch occurs.
     2. Output Links (`output`): Direct link to the nearest
     valid pattern
        ending in the failure chain (handles
        overlapping/substring patterns).
   - OPTIMIZATION: We convert the Trie into a Deterministic
   Finite Automaton
     (DFA). Transition `next[c]` either points to the actual
     child OR the
     fallback state in O(1).

   Complexity:
   - Build Time: O(Sum of pattern lengths * Alphabet Size)
   - Search Time: O(Length of Text + Number of Matches)
*/

const int ALPHABET = 26;
const char BASE_CHAR = 'a'; // Change to 'A' or '0' depending
on the problem

struct ACNode {
  int next[ALPHABET];
  int fail;
  int output; // Also known as dict_link (points to the
  nearest terminal node)
  vector<int> pattern_ids; // IDs of patterns that end exactly
  at this node

  ACNode() {
    fill(begin(next), end(next), 0);
    fail = 0;
    output = 0;
  }
};

struct AhoCorasick {
  vector<ACNode> t;

  AhoCorasick() {
    t.emplace_back(); // Root node at index 0
  }

  // 1. Insert a pattern into the Trie
  void insert(const string& s, int id) {
    int cur = 0;
    for (char c : s) {
      int v = c - BASE_CHAR;
      if (!t[cur].next[v]) {
        t[cur].next[v] = t.size();
        t.emplace_back();
      }
      cur = t[cur].next[v];
    }
    t[cur].pattern_ids.push_back(id);
  }

  // 2. Build Failure and Output links via BFS
  void build() {
    queue<int> q;

    // Initialize root's children
    for (int i = 0; i < ALPHABET; i++) {
      if (t[0].next[i]) {
        t[t[0].next[i]].fail = 0;
        q.push(t[0].next[i]);
      }
    }

    while (!q.empty()) {
      int u = q.front();
      q.pop();

      for (int i = 0; i < ALPHABET; i++) {
        int v = t[u].next[i];
        if (v) {
          // If child exists, set its failure link to the
          child of u's fail node
          t[v].fail = t[t[u].fail].next[i];

          // Optimization: Output link points to the nearest
          pattern end
          if (!t[t[v].fail].pattern_ids.empty()) {
            t[v].output = t[v].fail;
          } else {
            t[v].output = t[t[v].fail].output;
          }

          q.push(v);
        } else {
          // Trie Automaton Optimization: If child doesn't
          exist,
          // point directly to the fallback state. This makes
          text traversal O(1).
          t[u].next[i] = t[t[u].fail].next[i];
        }
      }
    }
  }

  // 3. Search for patterns in the given text
  void search(const string& text) {
    int cur = 0;
    for (int i = 0; i < text.length(); i++) {
      int c = text[i] - BASE_CHAR;
      cur = t[cur].next[c]; // O(1) transition due to
      Automaton optimization

      // Traverse output links to find all patterns ending at
      current position i
      int temp = cur;
      while (temp != 0) {
        for (int id : t[temp].pattern_ids) {
          // You can store or process the match here
          cout << "Pattern ID " << id << " ends at index " <<
          i << "\n";
        }
        // Jump to the next valid pattern in the failure chain
        // If the current node doesn't have an end word but
        has an output link,
        // this safely jumps to it.
        if (t[temp].pattern_ids.empty()) {
          temp = t[temp].output;
        } else {
          temp = t[temp].output;
        }
      }
    }
  }
};

/*
   USAGE EXAMPLE:

   signed main() {
     AhoCorasick ac;

     vector<string> patterns = {"he", "she", "his", "hers"};
     for (int i = 0; i < patterns.size(); i++) {
       ac.insert(patterns[i], i);
     }

     // MUST call build() after inserting all patterns and
     before searching
```

```
    ac.build();

    string text = "ushers";
    cout << "Searching in text: " << text << "\n";
    ac.search(text);

    // Output expected:
    // Pattern ID 1 ("she") ends at index 3
    // Pattern ID 0 ("he") ends at index 3
    // Pattern ID 3 ("hers") ends at index 5

    return 0;
  }
*/
```

# 6 Math

## 6.1 Extended Euclid

```
// 1. Core Extended GCD
// Returns gcd(a, b) and updates x, y via reference.
long long extgcd(long long a, long long b, long long &x, long
long &y) {
  if (b == 0) {
    x = 1;
    y = 0;
    return a;
  }
  long long x1, y1;
  long long d = extgcd(b, a % b, x1, y1);
  x = y1;
  y = x1 - y1 * (a / b);
  return d;
}

// 2. Solve Linear Diophantine Equation: ax + by = c
// Returns true if a solution exists, false otherwise.
// If true, (x0, y0) stores one valid base solution, and g
stores gcd(a, b).
//
// GENERAL SOLUTION GENERATOR:
// All solutions can be generated using an integer k:
// x = x0 + k * (b / g)
// y = y0 - k * (a / g)
bool solve_diophantine(long long a, long long b, long long c,
long long &x0, long long &y0, long long &g) {
  // Solve for abs(a) and abs(b) first to maintain correct
  signs
  g = extgcd(abs(a), abs(b), x0, y0);

  if (c % g != 0) {
    return false; // No solution if c is not divisible by
    gcd(a, b)
  }

  x0 *= c / g;
  y0 *= c / g;

  // Adjust signs back
  if (a < 0) x0 = -x0;
  if (b < 0) y0 = -y0;

  return true;
}

// 3. Modular Multiplicative Inverse
// Solves: ax = 1 (mod m).
// Valid ONLY IF a and m are coprime (gcd(a, m) == 1).
// Returns -1 if the inverse does not exist.
long long mod_inverse(long long a, long long m) {
  long long x, y;
  long long g = extgcd(a, m, x, y);

  if (g != 1) {
    return -1; // Inverse doesn't exist
  }

  // x can be negative, so we adjust it to be in the range [0,
  m - 1]
  return (x % m + m) % m;
}
```

## 6.2 CRT

```
/*
   GENERALIZED CHINESE REMAINDER THEOREM (CRT)
   Idea:
   - Solves a system of linear congruences:
       x = a_1 (mod m_1)
       x = a_2 (mod m_2)
       ...
       x = a_n (mod m_n)
   - Unlike the standard CRT, this GENERALIZED version does
   NOT require
     the moduli (m_i) to be pairwise coprime.
   - It works by iteratively merging two congruences into one.
   Math:
   - Merging x = a1 (mod m1) and x = a2 (mod m2):
     We need x = a1 + k1 * m1 = a2 + k2 * m2
     => k1 * m1 - k2 * m2 = a2 - a1
     This is a linear Diophantine equation. Let g = gcd(m1,
     m2).
     If (a2 - a1) is not divisible by g, there is NO SOLUTION.
     Otherwise, we solve for k1 and find the new merged
     congruence:
     x = new_a (mod lcm(m1, m2)).
   Complexity: O(N log(LCM))
   Note: Uses __int128_t to prevent overflow during
   intermediate multiplications.
*/
long long extgcd(long long a, long long b, long long &x, long
long &y) {
  if (b == 0) {
    x = 1;
    y = 0;
    return a;
  }
  long long x1, y1;
  long long d = extgcd(b, a % b, x1, y1);
  x = y1;
  y = x1 - y1 * (a / b);
  return d;
}

// Generalized CRT
// Inputs: vector of remainders `a`, vector of moduli `m`.
// Returns: A pair {ans_a, ans_m} such that x = ans_a (mod
ans_m).
// If no solution exists, returns {-1, -1}.
pair<long long, long long> crt(const vector<long long>& a,
const vector<long long>& m) {
  assert(a.size() == m.size());
  if (a.empty()) return {0, 1}; // Trivial case

  long long ans_a = a[0];
  long long ans_m = m[0];

  for (int i = 1; i < (int)a.size(); i++) {
    long long cur_a = a[i];
    long long cur_m = m[i];

    long long u, v;
    long long g = extgcd(ans_m, cur_m, u, v);

    // Check if a solution exists for this pair
    if ((cur_a - ans_a) % g != 0) {
      return {-1, -1}; // No solution
    }

    long long lcm = (ans_m / g) * cur_m;

    // Calculate k1 = (a2 - a1) / g * u (mod m2 / g)
    // Using __int128_t to avoid overflow when multiplying
    (cur_a - ans_a) / g and u
    long long diff = cur_a - ans_a;
    __int128_t k = (__int128_t)diff / g * u;
    k %= (cur_m / g);

    // Substitute k back to find the new remainder
    ans_a = ans_a + (long long)k * ans_m;

    // Normalize ans_a to be positive and within [0, lcm - 1]
```

```
    ans_a = (ans_a % lcm + lcm) % lcm;
    ans_m = lcm;
  }

  return {ans_a, ans_m};
}

/*
   USAGE EXAMPLE:

   signed main() {
     // System of equations:
     // x = 2 (mod 3)
     // x = 3 (mod 5)
     // x = 2 (mod 7)

     vector<long long> a = {2, 3, 2};
     vector<long long> m = {3, 5, 7};

     pair<long long, long long> res = crt(a, m);

     if (res.first == -1) {
       cout << "No solution exists.\n";
     } else {
       cout << "Smallest positive solution: x = " << res.first
       << "\n";
       cout << "General solution: x = " << res.first << " (mod
       " << res.second << ")\n";
     }

     // Output:
     // Smallest positive solution: x = 23
     // General solution: x = 23 (mod 105)

     return 0;
   }
*/
```

## 6.3 Lucas Theorem

```
/*
   LUCAS THEOREM
   Idea:
   - Computes nCr(n, k) % p where n and k can be very large
   (up to 10^18),
     but p is a relatively small PRIME number (e.g., p <=
     10^6).
   - Theorem: nCr(n, k) = Product of nCr(n_i, k_i) (mod p),
   where n_i and
     k_i are the digits of n and k in base p.
   - If at any point k_i > n_i, then nCr(n_i, k_i) = 0,
   meaning the entire
     result becomes 0.
   Complexity:
   - Precomputation: O(p) to compute factorials up to p - 1.
   - Query: O(log_p(n)) per nCr query.
   - Space: O(p)
   Requirement:
   - p MUST be a prime number.
*/
struct LucasTheorem {
  long long p;
  vector<long long> fact, invFact;

  long long power(long long base, long long exp) {
    long long res = 1;
    base %= p;
    while (exp > 0) {
      if (exp % 2 == 1) res = (res * base) % p;
      base = (base * base) % p;
      exp /= 2;
    }
    return res;
  }

  long long modInverse(long long n) {
    return power(n, p - 2);
  }

  // Constructor: Precomputes factorials and inverse
  factorials up to p - 1
  LucasTheorem(long long prime_mod) {
    p = prime_mod;
    fact.assign(p, 1);
    invFact.assign(p, 1);

    for (int i = 1; i < p; i++) {
      fact[i] = (fact[i - 1] * i) % p;
    }

    invFact[p - 1] = modInverse(fact[p - 1]);
    for (int i = p - 2; i >= 0; i--) {
      invFact[i] = (invFact[i + 1] * (i + 1)) % p;
    }
  }

  // Computes nCr(n, k) % p for small values (n, k < p)
  long long nCr_small(long long n, long long k) {
    if (k < 0 || k > n) return 0;
    long long num = fact[n];
    long long den = (invFact[k] * invFact[n - k]) % p;
    return (num * den) % p;
  }

  // Computes nCr(n, k) % p for large values using Lucas
  Theorem
  long long lucas(long long n, long long k) {
    if (k < 0 || k > n) return 0;

    long long res = 1;
    while (n > 0 || k > 0) {
      long long ni = n % p;
      long long ki = k % p;

      if (ki > ni) return 0;

      res = (res * nCr_small(ni, ki)) % p;

      n /= p;
      k /= p;
    }
    return res;
  }
};
```

## 6.4 Euler Totient

```
/*
   EULER'S TOTIENT FUNCTION (PHI FUNCTION)
   Idea:
   - phi(n) counts the number of integers between 1 and n
   inclusive
     that are coprime to n (i.e., gcd(x, n) == 1).
   Important Properties for CP:
   1. If p is prime: phi(p) = p - 1
   2. If p is prime and k >= 1: phi(p^k) = p^k - p^(k-1)
   3. Multiplicative: If gcd(a, b) == 1, then phi(a * b) =
   phi(a) * phi(b)
   4. Euler's Theorem: If gcd(a, m) == 1, then a^phi(m) = 1
   (mod m)
   5. Sum of phi over divisors: The sum of phi(d) for all
   divisors d of n
      is exactly equal to n. (Sum_{d|n} phi(d) = n)
*/
// 1. Single Computation
// Computes phi(n) in O(sqrt(n)) time.
// Uses the formula: phi(n) = n * (1 - 1/p1) * (1 - 1/p2) *
...
// where p_i are the distinct prime factors of n.
long long get_phi(long long n) {
  long long result = n;
  for (long long i = 2; i * i <= n; i++) {
    if (n % i == 0) {
      while (n % i == 0) {
        n /= i;
      }
      result -= result / i;
    }
  }
  if (n > 1) {
    result -= result / n;
  }
```

```
  return result;
}

// 2. Sieve Computation (Batch Computation)
// Computes phi(i) for all 1 <= i <= N in O(N log log N) time.
// Similar to the Sieve of Eratosthenes.
vector<int> phi_sieve(int n) {
  vector<int> phi(n + 1);

  for (int i = 0; i <= n; i++) {
    phi[i] = i;
  }

  for (int i = 2; i <= n; i++) {
    if (phi[i] == i) {
      for (int j = i; j <= n; j += i) {
        phi[j] -= phi[j] / i;
      }
    }
  }
  return phi;
}

/*
   USAGE EXAMPLE:
   signed main() {
     // 1. Single query for a large number
     long long n = 12;
     cout << "phi(" << n << ") = " << get_phi(n) << "\n";
     // Output: 4 (Coprimes are 1, 5, 7, 11)
     // 2. Precomputing phi for all numbers up to MAX_N
     int MAX_N = 100000;
     vector<int> phi = phi_sieve(MAX_N);
     cout << "phi(10) = " << phi[10] << "\n"; // Output: 4
     cout << "phi(97) = " << phi[97] << "\n"; // Output: 96
     (Since 97 is prime)
     return 0;
   }
*/
```

## 6.5 Mobius Function

```
/*
   MOBIUS FUNCTION & INVERSION

   Definition of mu(n):
   - mu(n) = 1  if n = 1
   - mu(n) = 0  if n has a squared prime factor (e.g., 4, 9,
   12, 18)
   - mu(n) = 1  if n is square-free with an EVEN number of
   prime factors
   - mu(n) = -1 if n is square-free with an ODD number of
   prime factors

   Important Properties for CP:
   1. Sum over divisors: The sum of mu(d) for all divisors d
   of n is 1
      if n = 1, and 0 otherwise. (Sum_{d|n} mu(d) = [n == 1])
      -> This is the most used property to replace [gcd(i, j)
      == 1].
   2. Mobius Inversion Formula:
      If g(n) = Sum_{d|n} f(d), then f(n) = Sum_{d|n} mu(d) *
      g(n/d).
*/
// 1. Single Computation
// Computes mu(n) in O(sqrt(n)) time.
int get_mu(long long n) {
  if (n == 1) return 1;
  int prime_count = 0;

  for (long long i = 2; i * i <= n; i++) {
    if (n % i == 0) {
      int count = 0;
      while (n % i == 0) {
        count++;
        n /= i;
      }
      if (count > 1) return 0;
      prime_count++;
    }
  }
  if (n > 1) {
    prime_count++;
  }

  return (prime_count % 2 == 1) ? -1 : 1;
}

// 2. Linear Sieve Computation (Batch Computation)
// Computes mu(i) for all 1 <= i <= N in exactly O(N) time.
struct MobiusSieve {
  int n;
  vector<int> mu;
  vector<int> primes;
  vector<bool> is_prime;

  MobiusSieve(int _n) {
    n = _n;
    mu.assign(n + 1, 0);
    is_prime.assign(n + 1, true);

    is_prime[0] = is_prime[1] = false;
    mu[1] = 1;

    for (int i = 2; i <= n; i++) {
      if (is_prime[i]) {
        primes.push_back(i);
        mu[i] = -1; // Prime numbers have exactly 1 prime
        factor (ODD)
      }
      for (int p : primes) {
        if (i * p > n) break;
        is_prime[i * p] = false;

        if (i % p == 0) {
          // i * p is divisible by p^2
          mu[i * p] = 0;
          break;
        } else {
          // p is a new distinct prime factor
          mu[i * p] = -mu[i];
        }
      }
    }
  }
};

/*
   USAGE EXAMPLE:

   signed main() {
     // 1. Single query for a large number
     long long n = 30; // 30 = 2 * 3 * 5 (3 prime factors ->
     odd -> mu = -1)
     cout << "mu(" << n << ") = " << get_mu(n) << "\n"; //
     Output: -1

     long long m = 12; // 12 = 2^2 * 3 (has squared prime
     factor 2^2 -> mu = 0)
     cout << "mu(" << m << ") = " << get_mu(m) << "\n"; //
     Output: 0

     // 2. Precomputing mu for all numbers up to MAX_N in O(N)
     int MAX_N = 100000;
     MobiusSieve sieve(MAX_N);

     cout << "mu(10) = " << sieve.mu[10] << "\n"; // 10 = 2 *
     5 -> Output: 1
     cout << "mu(97) = " << sieve.mu[97] << "\n"; // 97 is
     prime -> Output: -1

     return 0;
   }
*/
```

## 6.6 Miller Rabin

```
/*
   MILLER-RABIN PRIMALITY TEST

   Idea:
```

```
   - A fast primality test. While originally probabilistic, it
   can be made
     DETERMINISTIC for all 64-bit integers by testing a
     specific, small set
     of prime bases.
   - To test if n is prime, we write n - 1 = d * 2^s, where d
   is odd.
   - For a base 'a', n is composite if a^d != 1 (mod n) AND
   a^(d * 2^r) != n - 1 (mod n)
     for all r in [0, s - 1].

   Bases for Deterministic Test:
   - For n < 2^64, testing the first 12 primes is strictly
   sufficient:
     {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}

   Complexity:
   - Time: O(K log n) where K is the number of bases (K <=
   12).
   - Space: O(1)
   - Uses __uint128_t to cleanly prevent overflow during
   64-bit multiplications.
*/
using u64 = uint64_t;
using u128 = __uint128_t;

// Modular multiplication using 128-bit integer to avoid
overflow
u64 mod_mul(u64 a, u64 b, u64 m) {
  return (u64)((u128)a * b % m);
}

// Modular exponentiation: (base^exp) % mod
u64 mod_pow(u64 base, u64 exp, u64 mod) {
  u64 res = 1;
  base %= mod;
  while (exp > 0) {
    if (exp % 2 == 1) res = mod_mul(res, base, mod);
    base = mod_mul(base, base, mod);
    exp /= 2;
  }
  return res;
}

// Miller-Rabin deterministic primality test for 64-bit
integers
bool is_prime(u64 n) {
  if (n < 2) return false;
  if (n == 2 || n == 3) return true;
  if (n % 2 == 0) return false;

  // Write n - 1 as d * 2^s
  u64 d = n - 1;
  int s = 0;
  while (d % 2 == 0) {
    d /= 2;
    s++;
  }

  // Deterministic bases for values up to 2^64
  static const u64 bases[] = {2, 3, 5, 7, 11, 13, 17, 19, 23,
  29, 31, 37};

  for (u64 a : bases) {
    // If n is smaller than or equal to the base, we are done
    // (since n is prime and has already passed the n % 2 == 0
    check,
    // it would just be one of the prime bases).
    if (n <= a) break;

    u64 x = mod_pow(a, d, n);
    if (x == 1 || x == n - 1) continue;

    bool composite = true;
    for (int r = 1; r < s; r++) {
      x = mod_mul(x, x, n);
      if (x == n - 1) {
        composite = false;
        break;
      }
    }
    if (composite) return false;
  }

  return true;
}

/*
   USAGE EXAMPLE:

   signed main() {
     long long n = 100000000000000003LL; // A large 64-bit
     number

     if (is_prime(n)) {
       cout << n << " is PRIME.\n";
     } else {
       cout << n << " is COMPOSITE.\n";
     }

     // Expected Output: 100000000000000003 is PRIME.

     return 0;
   }
*/
```

# 7 Geometry

## 7.1 Geomtry Point

```
// Point coordinates use ll (long long).
// Dot, cross and len2 require their intermediate results to
fit in ll.
struct Point {
  typedef ll T;
  T x, y;

  Point(T _x = 0, T _y = 0) : x(_x), y(_y) {}

  // Lexicographical order: x, then y
  bool operator<(Point p) const { return tie(x, y) < tie(p.x,
  p.y); }
  bool operator>(Point p) const { return tie(x, y) > tie(p.x,
  p.y); }
  bool operator==(Point p) const { return tie(x, y) ==
  tie(p.x, p.y); }
  bool operator!=(Point p) const { return tie(x, y) !=
  tie(p.x, p.y); }

  Point operator+(Point p) const { return Point(x + p.x, y +
  p.y); }
  Point operator-(Point p) const { return Point(x - p.x, y -
  p.y); }

  T operator*(Point p) const { return x * p.x + y * p.y; }  //
  Dot product
  T operator^(Point p) const { return x * p.y - y * p.x; }  //
  Cross product

  Point operator*(T d) const { return Point(x * d, y * d); }
  Point operator/(T d) const { return Point(x / d, y / d); }
  // Integer division

  T len2() const { return x * x + y * y; }
  double len() const { return hypot((double)x, (double)y); }

  Point perp() const { return Point(-y, x); }  // 90 degrees
  CCW

  friend ostream& operator<<(ostream &os, const Point &p) {
    return os << "(" << p.x << ", " << p.y << ")";
  }
};

ll sgn(ll x) {
  return (x > 0) - (x < 0);
}

// > 0: CCW; < 0: CW; = 0: collinear
ll ccw(const Point &p0, const Point &p1, const Point &p2) {
```

```
  return (p1 - p0) ^ (p2 - p0);
}
```

## 7.2 Convex Hull

```
// Andrew's monotone chain: O(N log N).
// Returns distinct extreme vertices in CCW order, without a
repeated first vertex.
// Collinear points on hull edges are removed; all-collinear
input -> 2 endpoints.
// For a hull containing EVERY boundary point, a different
collinear policy is needed.
vector<Point> convex_hull(vector<Point> dots) {
  sort(dots.begin(), dots.end());
  dots.erase(unique(dots.begin(), dots.end()), dots.end());
  if (dots.size() <= 1) return dots;

  vector<Point> A(1, dots[0]);
  const int n = (int)dots.size();
  for (int c = 0; c < 2; reverse(dots.begin(), dots.end()),
  c++) {
    for (int i = 1, t = (int)A.size(); i < n;
    A.emplace_back(dots[i++])) {
      while ((int)A.size() > t && ccw(A[A.size() - 2],
      A.back(), dots[i]) <= 0) {
        A.pop_back();
      }
    }
  }
  A.pop_back();
  return A;
}
```

## 7.3 Manhattan MST

```
// Manhattan MST candidate edges: {weight, u, v}, 0-based
point indices.
// Run Kruskal on these edges to obtain the MST. O(N log N)
time, O(N) edges.
// Coordinates, transforms and Manhattan distances must fit in
ll.
vector<array<ll, 3>> manhattan_mst(vector<Point> ps) {
  int n = (int)ps.size();
  vector<int> id(n);
  iota(id.begin(), id.end(), 0);
  vector<array<ll, 3>> edges;

  for (int k = 0; k < 4; k++) {
    // Sort by x + y. Use __int128 to avoid overflow in this
    comparison.
    sort(id.begin(), id.end(), [&](int i, int j) {
      return (__int128)ps[i].x + ps[i].y < (__int128)ps[j].x +
      ps[j].y;
    });

    map<ll, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y); it !=
      sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        Point d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.x + d.y, i, j});
      }
      sweep[-ps[i].y] = i;
    }

    for (Point &p : ps) {
      if (k & 1) {
        p.x = -p.x;
      } else {
        swap(p.x, p.y);
      }
    }
  }
  return edges;
}
```

## 7.4 Common Geometry Operations

```
// Requires Point, ccw and sgn from geometry_point.h.
// Integer dot/cross computations must fit in ll.
const ll LINF = 2e18;

// Inclusive: endpoints count. Also handles a degenerate
segment p0 == p1.
bool on_segment(const Point &p, const Point &p0, const Point
&p1) {
  return ccw(p0, p1, p) == 0
    && min(p0.x, p1.x) <= p.x && p.x <= max(p0.x, p1.x)
    && min(p0.y, p1.y) <= p.y && p.y <= max(p0.y, p1.y);
}

// Shortest distance from p to the closed segment [p0, p1].
double dist_segment(const Point &p, const Point &p0, const
Point &p1) {
  if ((p1 - p0) * (p - p1) >= 0) return (p - p1).len();
  if ((p0 - p1) * (p - p0) >= 0) return (p - p0).len();
  return abs((double)((p1 - p0) ^ (p - p0))) / (p1 -
  p0).len();
}

// Strict point-in-convex-polygon test in O(log N).
// Preconditions: strictly convex polygon, >= 3 vertices,
CLOCKWISE order,
// no repeated first vertex at the end. Boundary is outside.
// convex_hull() returns CCW, so reverse its output before
calling this.
bool inside_convex(const Point &p, const vector<Point> &poly)
{
  int n = (int)poly.size();
  if (n < 3) return false;
  if (ccw(poly[0], poly[1], p) >= 0) return false;
  if (ccw(poly[n - 1], poly[0], p) >= 0) return false;

  int l = 1, r = n - 1;
  while (l < r) {
    int mid = (l + r + 1) / 2;
    if (ccw(poly[0], p, poly[mid]) >= 0) {
      l = mid;
    } else {
      r = mid - 1;
    }
  }
  return ccw(poly[l], p, poly[l + 1]) > 0;
}

// Winding-number test for a simple polygon, CW or CCW, in
O(N).
// Return: 0 = outside, LINF = on boundary, otherwise strictly
inside.
// For a CCW simple polygon the interior winding number is
positive;
// for CW it is negative.
ll wn_poly(const Point &p, const vector<Point> &poly) {
  ll wn = 0;
  int n = (int)poly.size();

  for (int i = 0; i < n; i++) {
    const Point &a = poly[i], &b = poly[(i + 1) % n];
    if (on_segment(p, a, b)) return LINF;

    if (a.y <= p.y) {
      if (b.y > p.y && ccw(a, b, p) > 0) ++wn;
    } else {
      if (b.y <= p.y && ccw(a, b, p) < 0) --wn;
    }
  }
  return wn;
}

// Inclusive segment intersection: touching
endpoints/overlapping count.
// Handles degenerate segments (a == b or c == d).
bool segment_intersect(const Point &a, const Point &b, const
Point &c, const Point &d) {
  ll o1 = ccw(a, b, c), o2 = ccw(a, b, d);
  ll o3 = ccw(c, d, a), o4 = ccw(c, d, b);

  if (o1 == 0 && on_segment(c, a, b)) return true;
```

```
  if (o2 == 0 && on_segment(d, a, b)) return true;
  if (o3 == 0 && on_segment(a, c, d)) return true;
  if (o4 == 0 && on_segment(b, c, d)) return true;

  return sgn(o1) * sgn(o2) < 0 && sgn(o3) * sgn(o4) < 0;
}
```

# 8 Miscellaneous

## 8.1 Alien Trick

```
/*
   ALIEN'S TRICK (WQS BINARY SEARCH / LAGRANGE RELAXATION)
   Idea:
   - Solves DP problems with an "exactly K items/partitions"
   constraint.
   - We drop the K constraint and add a penalty `lambda` for
   each partition.
   - We binary search on `lambda`. If we use > K partitions,
   we increase
     the penalty. If we use < K partitions, we decrease it.
   - To handle collinear points on the convex hull, we always
   prefer the
     MAXIMUM (or MINIMUM) number of partitions for the same
     cost, and use
     `cnt >= K` in the binary search.
   Condition:
   - Let f(x) be the optimal cost using exactly x partitions.
   - f(x) MUST be convex or concave.
   Complexity: O(N * log(MAX_PENALTY)) instead of O(N * K).
*/
const long long LINF = 1e18;

int n;
vector<long long> a;

// 1. The 1D DP function without the 'K' constraint
// Returns a pair: {optimal_cost_with_penalty,
number_of_partitions_used}
pair<long long, int> solve_dp(long long lambda) {
  // dp[i] stores {min_cost, max_partitions_used} up to index
  i
  vector<pair<long long, int>> dp(n + 1, {LINF, 0});
  dp[0] = {0, 0};

  for (int i = 1; i <= n; i++) {
    for (int j = 0; j < i; j++) {
      // cost(j+1, i) is the cost of the segment from j+1 to i
      // We ADD `lambda` as a penalty for creating this new
      segment
      long long segment_cost = 0; /* TODO: Calculate real cost
      */
      long long new_cost = dp[j].first + segment_cost +
      lambda;
      int new_cnt = dp[j].second + 1;
      // We want to MINIMIZE cost.
      // TIE-BREAKER: If costs are equal, MAXIMIZE the number
      of partitions.
      if (new_cost < dp[i].first || (new_cost == dp[i].first
      && new_cnt > dp[i].second)) {
        dp[i] = {new_cost, new_cnt};
      }
    }
  }
  return dp[n];
}

/*
   USAGE EXAMPLE:
   signed main() {
     n = 1000;
     int K = 50; // We need exactly K partitions

     // Binary search for the optimal penalty `lambda`
     // The range depends on the maximum possible cost
     difference
     long long l = -1e12, r = 1e12;
     long long ans = -1;

     while (l <= r) {
       long long mid = l + (r - l) / 2;

       pair<long long, int> res = solve_dp(mid);
       long long cost = res.first;
       int cnt = res.second;

       // If we used >= K partitions, it means the penalty is
       small enough
       // to allow at least K partitions. We record the answer
       and try to
       // INCREASE the penalty to force fewer partitions.
       if (cnt >= K) {
         // The real cost is the DP cost MINUS the penalty we
         added for K partitions
         ans = cost - K * mid;
         l = mid + 1; // Increase penalty
       } else {
         r = mid - 1; // Decrease penalty
       }
     }

     cout << "Minimum cost using exactly " << K << "
     partitions: " << ans << "\n";
     return 0;
   }
*/
```

## 8.2 Interval Container

```
/*
   INTERVAL CONTAINER (DISJOINT INTERVALS SET)
   Idea:
   - Maintains a set of disjoint intervals [L, R].
   - Automatically merges overlapping or adjacent intervals
   when adding.
   - Automatically splits intervals when removing a segment.
   - Dynamically maintains the total length/count of covered
   points.
*/
const long long LINF = 2e18;

struct IntervalContainer {
  // Stores intervals as {L, R}.
  // R is inclusive. For example, [1, 3] covers points 1, 2,
  and 3.
  set<pair<long long, long long>> st;
  long long total_length = 0;

  // 1. Add an interval [L, R] and merge with
  overlapping/adjacent ones
  void add_interval(long long L, long long R) {
    if (L > R) return;
    auto it = st.lower_bound({L - 1, -LINF});
    if (it != st.begin()) {
      auto prev = it;
      --prev;
      if (prev->second >= L - 1) {
        it = prev;
      }
    }

    long long nl = L, nr = R;
    while (it != st.end() && it->first <= R + 1) {
      nl = min(nl, it->first);
      nr = max(nr, it->second);
      total_length -= (it->second - it->first + 1);
      it = st.erase(it); // C++11: erase returns the iterator
      to the next element
    }

    st.insert({nl, nr});
    total_length += (nr - nl + 1);
  }

  // 2. Remove an interval [L, R] (punch a hole)
  void remove_interval(long long L, long long R) {
    if (L > R) return;
    auto it = st.lower_bound({L, -LINF});
    if (it != st.begin()) {
      auto prev = it;
      --prev;
```

```
      if (prev->second >= L) {
        it = prev;
      }
    }

    while (it != st.end() && it->first <= R) {
      long long il = it->first;
      long long ir = it->second;
      total_length -= (ir - il + 1);
      it = st.erase(it);

      if (il < L) {
        st.insert({il, L - 1});
        total_length += (L - 1 - il + 1);
      }

      if (ir > R) {
        st.insert({R + 1, ir});
        total_length += (ir - (R + 1) + 1);
      }
    }
  }

  // 3. Check if a specific point x is covered by any interval
  bool contains(long long x) {
    auto it = st.upper_bound({x, LINF});
    if (it == st.begin()) return false;
    --it;
    return it->second >= x;
  }

  // 4. Get total covered points length
  long long get_length() {
    return total_length;
  }
};

/*
   USAGE EXAMPLE:
   signed main() {
     IntervalContainer ic;

     ic.add_interval(1, 5);
     ic.add_interval(7, 10);
     cout << "Length: " << ic.get_length() << "\n"; // (5-1+1)
     + (10-7+1) = 5 + 4 = 9

     // Merge adjacent and overlapping segments
     // Adding [4, 8] will merge [1, 5] and [7, 10] into [1,
     10]
     ic.add_interval(4, 8);
     cout << "Length: " << ic.get_length() << "\n"; // Output:
     10

     // Punch a hole in the middle
     ic.remove_interval(3, 6);
     cout << "Length: " << ic.get_length() << "\n"; //
     Remainder: [1, 2] and [7, 10] -> Length: 6

     // Check points
     cout << "Contains 2? " << ic.contains(2) << "\n"; //
     Output: 1 (true)
     cout << "Contains 5? " << ic.contains(5) << "\n"; //
     Output: 0 (false)

     return 0;
   }
*/
```

## 8.3 Ternary Search

```
/*
   Idea:
   - Finds the minimum or maximum of a Unimodal function (a
   function that
     strictly increases then strictly decreases, or
     vice-versa).
   Two variants:
   1. Floating-point: Iterate a fixed number of times (e.g.,
   200) instead
      of testing (R - L > eps). 200 iterations shrink the
      search space by
      (2/3)^200, which is enough for any floating-point
      precision limit.
   2. Integer: Search until the range [L, R] is small (e.g.,
   length <= 3),
      then use a simple linear scan. This avoids infinite
      loops caused by
      integer division rounding.
*/
// The unimodal function we want to optimize.
// Example: f(x) = (x - 5)^2 + 10 (Minimum is at x = 5)
double f_double(double x) {
  return (x - 5.0) * (x - 5.0) + 10.0;
}

long long f_int(long long x) {
  return (x - 5) * (x - 5) + 10;
}

struct TernarySearch {
  // 1. Ternary Search on Real Numbers (Floating-point)
  // Returns the x-coordinate of the MINIMUM.
  // To find MAXIMUM: change `if (f_double(m1) <
  f_double(m2))` to `>`
  double solve_double(double L, double R) {
    // 200 iterations are ALWAYS enough to achieve maximum
    double precision
    for (int i = 0; i < 200; i++) {
      double m1 = L + (R - L) / 3.0;
      double m2 = R - (R - L) / 3.0;

      if (f_double(m1) < f_double(m2)) {
        R = m2; // The minimum is closer to m1, discard [m2,
        R]
      } else {
        L = m1; // The minimum is closer to m2, discard [L,
        m1]
      }
    }
    return L; // L and R are practically identical now
  }

  // 2. Ternary Search on Integers
  // Returns the x-coordinate of the MINIMUM.
  // To find MAXIMUM: change `<` to `>` in BOTH the ternary
  condition
  // and the final linear scan.
  long long solve_int(long long L, long long R) {
    // Stop when the range is small enough (3 points or fewer)
    while (R - L > 2) {
      long long m1 = L + (R - L) / 3;
      long long m2 = R - (R - L) / 3;

      if (f_int(m1) < f_int(m2)) {
        R = m2;
      } else {
        L = m1;
      }
    }

    // Linear scan for the remaining elements to avoid integer
    rounding bugs
    long long best_x = L;
    long long best_val = f_int(L);
    for (long long i = L + 1; i <= R; i++) {
      long long current_val = f_int(i);
      if (current_val < best_val) {
        best_val = current_val;
        best_x = i;
      }
    }

    return best_x;
  }
};
```